Software Engineering Project

DispatchNet:

Alessio Zanon
Leonardo Tonetta
Luca Fiorini

Use Case & Class Analysis

Contents

Readability notes

Bold words are system-important constructs.
Underlined nouns are actors of the system.
Italic words are special keywords to denote important limits of the project.
Hyperlinks in reference to sections and subsections will be blue.
Hyperlinks to external web content will be in magenta.
Green text is text in D1 added or modified for D2

Purpose

This document describes the use cases, their relation, and the class-based diagram of Project "TRE", which implements a rail transport management system.

The purpose of this document is to:

- Represent all intended use cases
- Relate use cases with eachother and with stakeholders
- Itemize and relate the major data-components employed by the system
- Provide a top-level view of the system's operations

Chapter 1

Context Description (D1)

Original, up-to-date file available at our Github here

O Objectives

A major European rail infrastructure manager received an idea for a unified rail service scheduling and ticketing system, and has chosen us to prototype a trial run, over multiple regions in their network.

Thus, the project provides a platform that covers the needs of both passengers and train companies, as well as an underlying management system of any regional-sized rail network, which supports network managers in its administration.
It provides a set of functionalities which support the planning of train travel. The system lets passengers discover paths, view scheduling and manage tickets. On the other side, it lets train companies introduce train services and network managers administer the network.
The project coordinates the functionalities for each stakeholder into a solution that has been identified in an application accessible for the aforesaid users.

The main objectives of the project are:

O.1 Ticketing system

Subsystem which deals with tickets, from the search to the possible actions after the purchase. It includes basic operations that the passenger can perform, such as:

- Viewing tickets details.
- Purchase.
- Accessing tickets history.
- Cancellation.

O.2 Route-finding

The system shall let passengers select a starting station, a destination station, and either a departure time or arrival time.
It computes and ranks possible **routes** that best fit the user's request and permissions (for example, not permitting cargo trains to passengers).
The user must be able to compare the options based on:

- Departure and arrival times.
- Number of transfers.
- Possible transfer locations.

O.3 Virtual board

The system provides passengers access to departure and arrival boards for any station. It also shows intermediate stations and scheduled times for each individual train service.

O.4 Rail system modeling

The system provides a data-driven approach to rail network modelling, supporting an efficient administration of the infrastructure.
It shall support:

- Management of the network by the network manager.

- Management of rolling stock by the network manager.
- Submission of a new service request by train companies.
- Review of a service request by the network manager.

Stakeholders

Internal

Human

Network managers, the product owner. They manage the rail network combining the availability of train companies with passengers' needs.
They are the primary stakeholders for making functional decisions.

Non-Human

Database, where all the information used by the application are stored.

External

Human

Passengers, the main end users. They want to streamline their interactions with the train transport system, including:

- Searching for routes and schedules.
- Analyzing departure/arrival boards as well as train information.
- Purchasing tickets.

Train companies, who provide the rolling stock. They request the introduction of services. They are also expected to provide information of trains they ask to be managed.
Anonymous users, who have not yet authenticated. They are considered as passengers with restricted access to functionalities that require authentication.

Non-human

Payment gateway, which provides the underlying payment methods.

Boundary: The project will not provide actual transactions nor integration with any payment application, instead a fake API.

FR Functional requirements

FR.1 Account classification

The system shall provide different functionalities, distinguishing users between between anonymous user, passenger, train company operator and network manager. The additional functionalities are described in Requirement restrictions based on user classification.
Clarified in which sense the system classifies users

FR.2 Account registration

Following FR.1, in reference to NFR.2, the system must allow anonymous users to register. Data must satisfy NFR.3. Password must satisfy NFR.4 and NFR.5.

FR.2.1 Consent management

Following NFR.6, in reference to FR.2, at registration the system must request the user's consent for personal data processing.

FR.2.2 Account registration notification

In reference to FR.2, after successful registration the system must notify the user as described at NFR.9.

FR.3 Account deletion

Following NFR.1, the system shall allow users to delete their accounts. Any other related data is also deleted.

FR.3.1 Account deletion notification

In reference to FR.3, after successful deletion the user is notified as described at NFR.9.

FR.4 Data request management

Following NFR.7, in reference to NFR.2, the system must provide users view of their stored information.

FR.5 Account data change

Following NFR.8, in reference to NFR.2, the system must allow users to change their data. New data must satisfy NFR.3.

FR.5.1 Account data change notification

In reference to FR.5, after a successful change operation the user is notified as described at NFR.9.

FR.6 Password reset

Following NFR.8, in reference to NFR.2, the system shall allow users to reset the password and change it (NFR.9). New password must satisfy NFR.4 and NFR.5.

FR.7 Login

Following FR.1, the system must let users login using their username and password defined at registration (FR.2).

Boundary: The network manager has default account username "admin" and password "Password1234".

FR.8 Logout

Following FR.1, the system shall let logged in users log out, switching to the restricted anonymous user's functionalities.

FR.9 Tickets details display

Following O.1, the system must allow passengers to view tickets.

FR.10 Tickets purchase

Following O.1, the system must allow passengers to buy tickets.

FR.10.1 Tickets purchase confirmation

Following O.1, in reference to FR.10, the passenger must be asked for a final confirmation before purchase.

FR.10.2 Tickets purchase notification

Following O.1, in reference to FR.10, at the end of a purchase operation, the system must notify the passenger as described at NFR.9.

FR.11 Tickets cancellation

Following O.1, the system must allow the cancellation of purchased tickets.

FR.11.1 Tickets cancellation confirmation

Following O.1, in reference to FR.11 and NFR.9, the passenger must be asked for a final confirmation before cancellation.

FR.11.2 Tickets cancellation notification

Following O.1, in reference to FR.11, at the end of a cancellation operation, the system must notify the passenger as described at NFR.9.

FR.12 Tickets history

Following O.1, the system must allow passengers to view their tickets history, both purchased and cancelled.

FR.12.1 Tickets classification

Following O.1, in reference to FR.12, the tickets history must show whether a ticket is valid, expired, or cancelled.

FR.13 Path search

Following O.2, the system needs to be able to compute multiple paths between stations , that is an ordered sequence of services from a departure to an arrival station.
Clarified what is a "path"

FR.13.1 Path points

Following O.2, extending FR.13, the passenger must be able to choose both the first and last stations of paths to be computed.

FR.13.2 Path timing

Following O.2, extending FR.13, the passenger shall be able to optionally specify a departure time and/or a latest arrival time. If neither is specified, the system must consider paths with departure times starting from the current time.

FR.14 Path ranking

Following O.2, in reference to FR.13, the system needs to rank the paths with a criteria selected by the passenger among a fixed list.

FR.14.1 Ranking length

Following O.2, in reference to FR.14, the system shall rank paths by prioritizing those with shortest travel time, and, secondarily, those with fewer intermediate stations.

FR.14.2 Ranking deviation "Departure"

Following O.2, in reference to FR.14, the system shall rank paths by prioritizing those whose first departure time is closest (but not earlier than) the user requested departure time.

FR.14.3 Ranking deviation 2 "Arrival"

Following O.2, in reference to FR.14, the system shall rank paths by prioritizing those whose last arrival time is closest (but not later than) the user requested departure time.

FR.14.4 Ranking cost

Following O.2, in reference to FR.14, the system shall rank paths by prioritizing those with lower total cost of tickets not already owned by the user.

FR.15 Transfers

Following O.2, in reference to FR.13, the system needs to compute transfer paths out of intersecting routes.

FR.16 Train access restriction

Following O.2, in reference to FR.13, the system must not consider train routes that pass by trains that the passenger is not allowed to purchase a ticket for, such as cargo trains or trains with unavailable seats.
Clarified which trains the passenger cannot access

FR.17 Station display

Following O.3, the system must provide departure and arrival times of all trains that pass through any station.

FR.17.1 Station search

Following O.3, the system must allow users to search such information by a station's name.
Added to actually allow users to request station information

FR.18 Train display

Following O.3, the system must provide departure and arrival times of a train at all the stations on its schedule.

FR.18.1 Train search

Following O.3, the system must allow user to search such information by a train's identifier.
Added to actually allow users to request train information

FR.19 Path display

The system must provide the board described at FR.18 for each train in a given path, limited to the segments in which the train participates.

FR.20 Creating a Service Request

Following O.4, companies must be allowed to send the system a request for the introduction of a service.

FR.21 Defining a Train Type

Following O.4, in reference to FR.22, the system must allow companies to register a train type to be used when defining a service.

FR.22 Defining a Service Type

Following O.4, in reference to FR.20 and FR.21. The system must allow companies to define services and classes, as well as a pricing model for passenger trains.

FR.23 Viewing the Network

Following O.3, the system must allow users to access a map of the network.

FR.24 Managing a Station

Following O.4, in reference to NFR.14, the system must allow network managers to manage a station.

FR.24.1 Registering a Station

Following O.4, in reference to FR.24, the system must allow network managers to register a station.

FR.24.2 Modifying a Station

Following O.4, in reference to FR.24, the system must allow network managers to modify any existing station.

FR.24.3 Deleting a Station

Following O.4, in reference to FR.24, the system must allow network managers to delete any existing station.

FR.25 Managing a Line

Following O.4, in reference to NFR.14, the system must allow network managers to manage a line.

FR.25.1 Registering a Line

Following O.4, in reference to FR.25, the system must allow network managers to register a line.

FR.25.2 Modifying a Line

Following O.4, in reference to FR.25, the system must allow network managers to modify any existing line.

FR.25.3 Deleting a Line

Following O.4, in reference to FR.25, the system must allow network managers to delete any existing line.

FR.26 Managing a Junction

Following O.4, in reference to NFR.14, the system must allow network managers to manage a junction.

FR.26.1 Registering a Junction

Following O.4, in reference to FR.26, the system must allow network managers to register a junction.

FR.26.2 Modifying a Junction

Following O.4, in reference to FR.26, the system must allow network managers to modify any existing junction.

FR.26.3 Deleting a Junction

Following O.4, in reference to FR.26, the system must allow network managers to delete any existing junction.

FR.27 Automated Service Integrity Checks

Following O.4, the system must autonomously perform integrity checks on proposed services. Integrity checks are defined by the following.

FR.27.1 Single Track Line Availability Check

Following O.4, specifying FR.27, the system must ensure that no two services are scheduled to traverse a single track line in opposite directions at the same time.

FR.27.2 Platform Availability Check

Following O.4, specifying FR.27, the system must ensure that no two services are scheduled to stop at the same platform at the same time.

FR.27.3 Track-side Systems Compatibility Check

Following O.4, specifying FR.27, the system must ensure that all trains used for a service are compatible with the signaling and electrification systems installed on the traversed infrastructure.
Boundary: the system will not implement track-side compatibility check

FR.27.4 Track Geometry Check

Following O.4, specifying FR.27, the system must ensure that all trains used for a service are compatible with both the loading and track gauges of the infrastructure they traverse.
Boundary: the system will not implement geometry check

FR.28 Displaying Integrity Check Results

Following O.4, extending FR.27, the system must provide the integrity check results.

FR.29 Service Request Review

Following O.4, in reference to FR.20, the system must allow the network manager to review requests.

FR.29.1 Service Request approval

Following O.4, in reference to FR.20 and FR.29, the network manager must be able to approve a service request, notifying (NFR.9) the train company that submitted the request.

FR.29.2 Service Request amendment

Following O.4, in reference to FR.20 and FR.29, the system must allow the network manager to send back an amended version of the request for the train company to review, notifying (NFR.9) them.
Boundary: the system will not implement any amendment functionalities.

FR.29.3 Service Request denial

Following O.4, in reference to FR.20 and FR.29, the system must allow the network manager to deny a service request, notifying (NFR.9) the train company that submitted the request.

Requirement restrictions based on user classification

Requirement	Anonymous	Passenger	Train company	Manager
FR.1 FR.23	Yes	Yes	Yes	Yes
FR.2 FR.6-7	Yes			
FR.3-5		Yes	Yes	
FR.8		Yes	Yes	Yes
FR.9 FR.13-19	Yes	Yes		
FR.10-12		Yes		
FR.20-22			Yes	
FR.24-28				Yes
FR.29			Self	Self

Caption:
Self: access to resources regarding only the personal part of the functionality.
Moved FR.6 from first row, because it was originally intended in this way, an authenticated user change password with FR5.

NFR Non-functional requirements

NFR.1 Data Protection Regulation

External requirement - Legislative requirement

The system must respect the Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data. With special attention to:

Art. 5 Principles relating to processing of personal data, ensuring ethical processing.

Art. 6, 7 Lawfulness of processing and Conditions for consent, both concerning consent for the processing of personal data.

Art. 13 Information to be provided where personal data are collected from the data subject, i.e. providing information of the controller of the processing.

Art. 15 Right of access by the data subject, i.e. users must be able to see their stored information.

Art. 16 Right to rectification, i.e. users must be able to change their information.

Art. 17 Right to erasure ('right to be forgotten').

Art. 32 Security of processing.

NFR.2 Account data

Product requirement - Security requirement

Following FR.1 and NFR.1, in reference to FR.2, user accounts must include username, email and password.

NFR.3 Account data details

Product requirement - Security requirement

Following NFR.1 and NFR.2, account data must satisfy the following:

- The username must be unique.
- The email must be unique.

NFR.4 Password security

Product requirement - Security requirement

Following NFR.1 and NFR.2, in reference to FR.2.1, the password must be at least 8 characters, with at least one upper and one lowercase letters and one digit.

NFR.5 Typo prevention

Product requirement - Security requirement

When a password is chosen or modified, the system must take prevention to mitigate typos: it must require the password to be typed twice and only accepted if both match.

NFR.6 Consent for personal data processing

External requirement - Legislative requirement

Following NFR.1, the system must comply with the GDPR for data processing consent.

NFR.7 Request of processed data

External requirement - Legislative requirement

Following NFR.1, the system must comply with the GDPR for accessing processed data.

NFR.8 Rectification of processed data

External requirement - Legislative requirement

Following NFR.1, the system must comply with the GDPR for the right of rectification.

NFR.9 Email notification

Product requirement - Usability requirement

In order to notify users, the system shall use the email service, using the one inserted at registration (FR.2).

NFR.10 Station-train display switching

Product requirement - Usability requirement
Users must be able to switch with a single prompt/input between the following

NFR.10.1 Station to train display

A station's display and a train's display passing through that station.

NFR.10.2 Train to station display

A train's display and a station's display in its schedule.

NFR.11 Station-path searching switching

Product requirement - Usability requirement
Users must be able to switch with a single prompt/input between the following.

NFR.11.1 Station "from" path

A station's display and the route-finding procedure with the station automatically set as the path's start.

NFR.11.2 Station "to" path

A station's display and the route-finding procedure with the station automatically set as the destination.

NFR.12 Path list to path list

Product requirement - Usability requirement

Users must be able to switch from a list of paths according to a ranking option to another, in no more steps/inputs than the ones required to select a different ranking.

NFR.13 Ranking non-choice

Product requirement - Usability requirement

The system shall default to the ranking described at FR.14.1 until a different ranking choice is given by the user.

NFR.14 Network infrastructure

Organizational requirement - Development requirement
The system structures the network infrastructure with junctions, stations and lines; in a way similar to a graph, where junctions and stations are nodes and lines are edges.
Added to explain what are the components of a network and how they interact.

Chapter 2

Use Cases & UML (D2)

RE: Objectives

A major European rail infrastructure manager received an idea for a unified rail service scheduling and ticketing system, and has chosen us to prototype a trial run, over multiple regions in their network.

Thus, the project provides a platform that covers the needs of both passengers and train companies, as well as an underlying management system of any regional-sized rail network, which supports network managers in its administration.
It provides a set of functionalities which support the planning of train travel. The system lets passengers discover paths, view scheduling and manage tickets. On the other side, it lets train companies introduce train services and network managers administer the network.
The project coordinates the functionalities for each stakeholder into a solution that has been identified in an application accessible for the aforesaid users.

The main objectives of the project are:

Ticketing system

Subsystem which deals with tickets, from the search to the possible actions after the purchase. It includes basic operations that the passenger can perform, such as:

- Viewing tickets details.
- Purchase.
- Accessing tickets history.
- Cancellation.

Route-finding

The system shall let passengers select a starting station, a destination station, and either a departure time or arrival time.
It computes and ranks possible **routes** that best fit the user's request and permissions (for example, not permitting cargo trains to passengers).
The user must be able to compare the options based on:

- Departure and arrival times.
- Number of transfers.
- Possible transfer locations.

Virtual board

The system provides passengers access to departure and arrival boards for any station. It also shows intermediate stations and scheduled times for each individual train service.

Rail system modeling

The system provides a data-driven approach to rail network modelling, supporting an efficient administration of the infrastructure.
It shall support:

- Management of the network by the network manager.
- Management of rolling stock by the network manager.
- Submission of a new service request by train companies.
- Review of a service request by the network manager.

RE: Stakeholders

Internal

Human

Network managers:, the product owner. They manage the rail network combining the availability of train companies with passengers' needs.
They are the primary stakeholders for making functional decisions.

Non-Human

Database, where all the information used by the application are stored.

External

Human

Passengers, the main end users. They want to streamline their interactions with the train transport system, including:

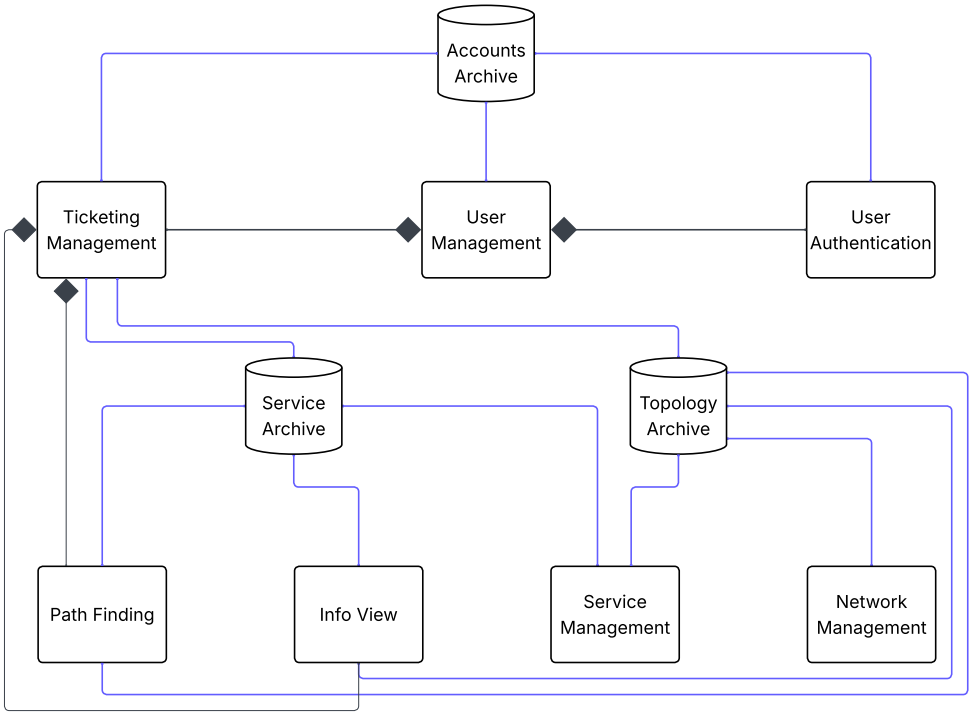
- Searching for routes and schedules.
- Analyzing departure/arrival boards as well as train information.
- Purchasing tickets.

Train companies, who provide the rolling stock. They request the introduction of services. They are also expected to provide information of trains they ask to be managed.
Anonymous users, who have not yet authenticated. They are considered as passengers with restricted access to functionalities that require authentication.

Non-human

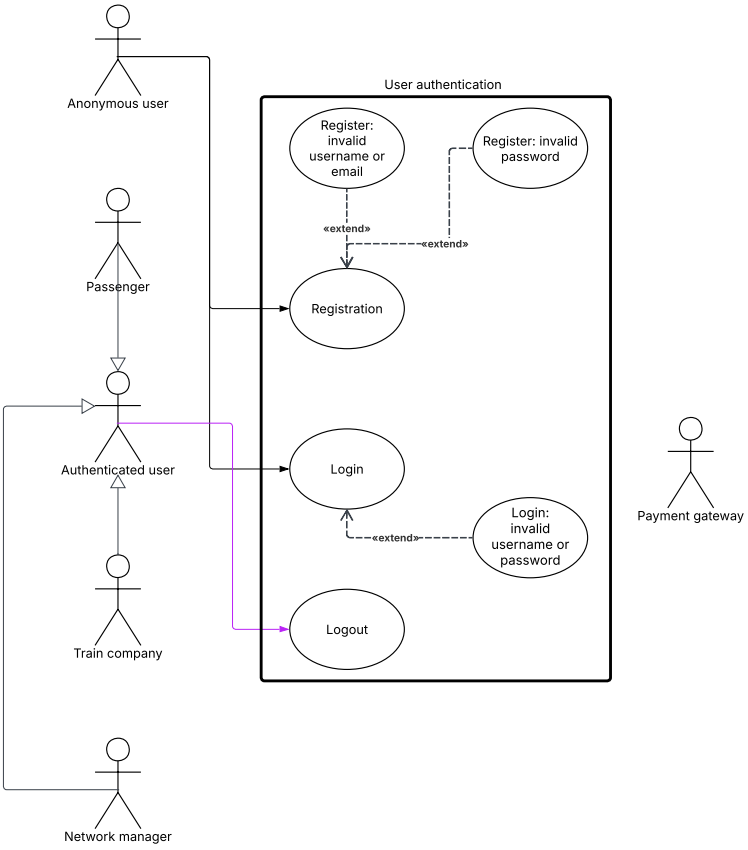
Payment gateway, which provides the underlying payment methods.

Component Diagram



Original image can be found on our Github here

User authentication



Original image can be found on our Github here

Registration

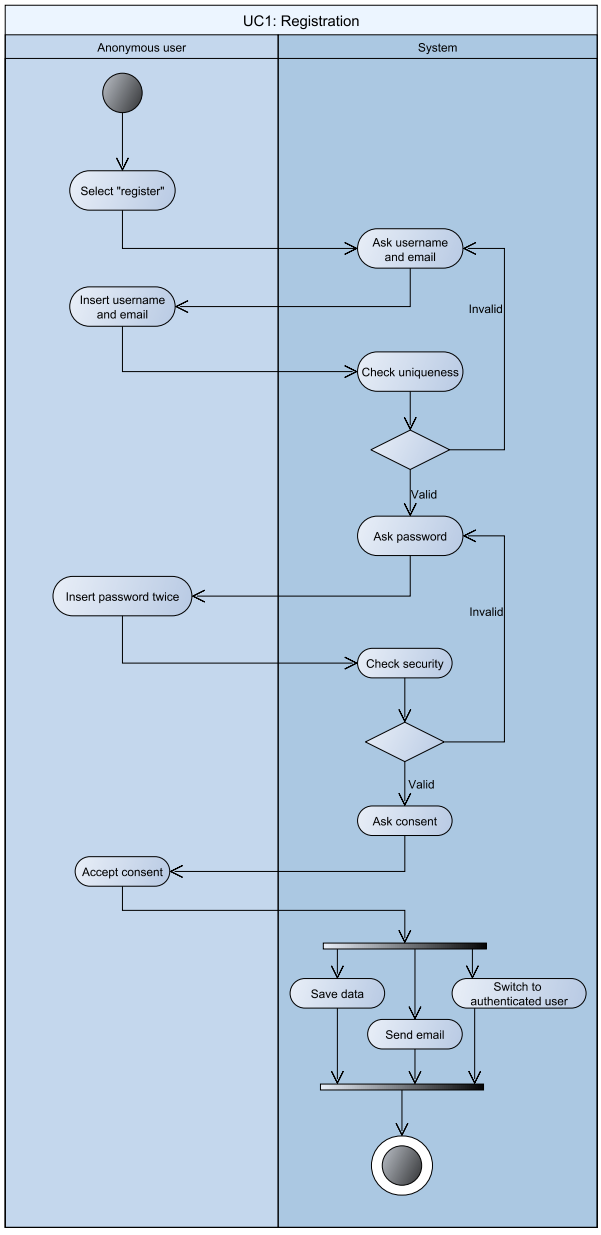
Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

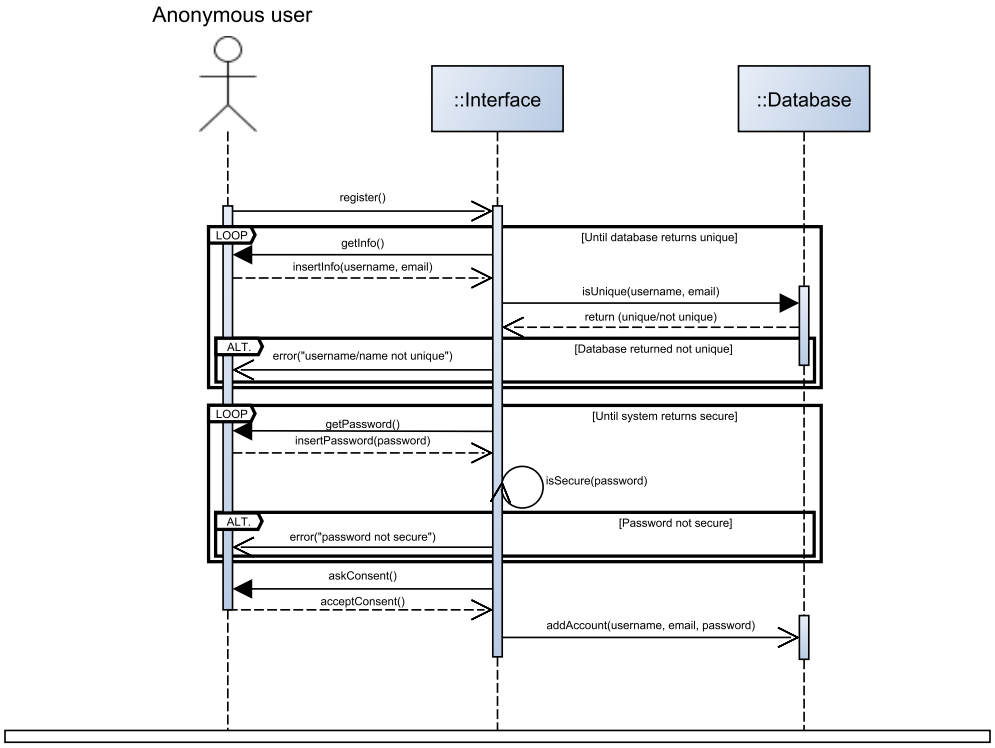
Use case	Registration		
Release (id)	UC1, v0.1, 19/04/2026		
Context of Use	User authentication and account management.		
Scope	Let an anonymous user create a new account. FR2 NFR1, NFR2, NFR3, NFR4, NFR5, NFR6, NFR9		
Level	Primary task		
Primary actor	Anonymous user		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Anonymous user	Wants to create an account	
	System	Manages the user interface, the database and their interactions	
Preconditions	The user must be logged out.		
Minimal Guarantees	No duplicated or unsecure account is created.		
Success Guarantees	The anonymous user successfully created an account and is logged in.		
Trigger	The anonymous user selects “register”.		
Description	**Step**	**Action**	
	1	The anonymous user selects the functionality “register”.	
	2	The system asks the user to insert username and email.	
	3	The user inserts the required data.	
	4	The system checks whether an account with at least one between username and email already exists.	
	5	The system asks the user to insert the password twice.	
	6	The user inserts the required data.	
	7	The system checks whether the password satisfies the security requirements and if the two passwords coincide.	
	8	The system asks the user’s consent for personal data processing.	
	9	The anonymous user accepts the personal data processing.	
	10	The system saves the data in the account archive.	

	11	The system notifies the user of successful registration with an email.
	12	The system switches to the authenticated user functionalities.
Extension	**Step**	**Other action**
	4a	Invalid username or email: UC1.1
	7a	Invalid password: UC1.2
Technology & data Variations	None	

Alternative flow	Registration: Invalid password	
Release (id)	UC1.2, v0.2, 25/04/2026	
Context of Use	User authentication and account management.	
Scope	The system alerts the anonymous user that they have inserted an invalid password. FR2 NFR1, NFR2, NFR4	
Level	Subfunction	
Primary actor	Anonymous user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous user	Wants to create an account
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted a password that does not satisfy the security requirements or the two passwords do not coincide.	
Minimal Guarantees	No unsecure account is created.	
Success Guarantees	None	
Trigger	UC1	
Description (Main Success Scenario)	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid password, specifying the problem.

	2	The sequence of events starts at step 5 of the main flow (UC1).
Technology & data Variations	None	


UC1: Registration
Anonymous user
System
Select "register"
Ask username and email
Insert username and email
Invalid
Check uniqueness
Valid
Ask password
Insert password twice
Invalid
Check security
Valid
Ask consent
Accept consent
Save data
Switch to authenticated user
Send email



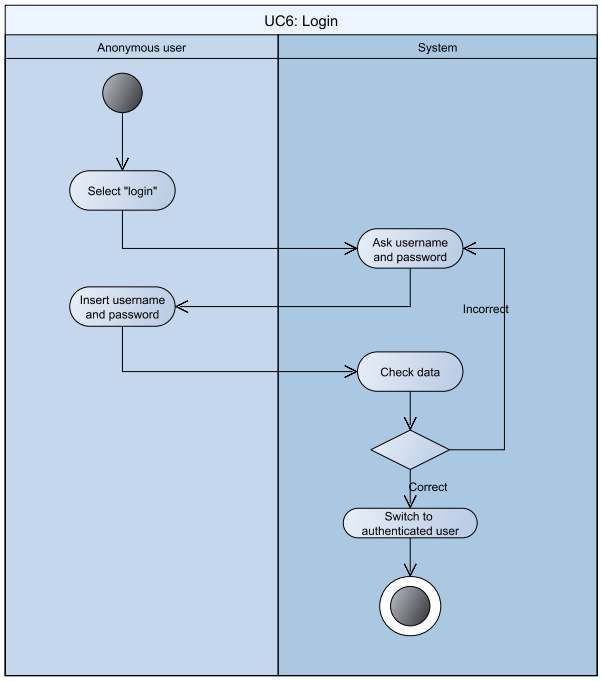
UC1: Registration

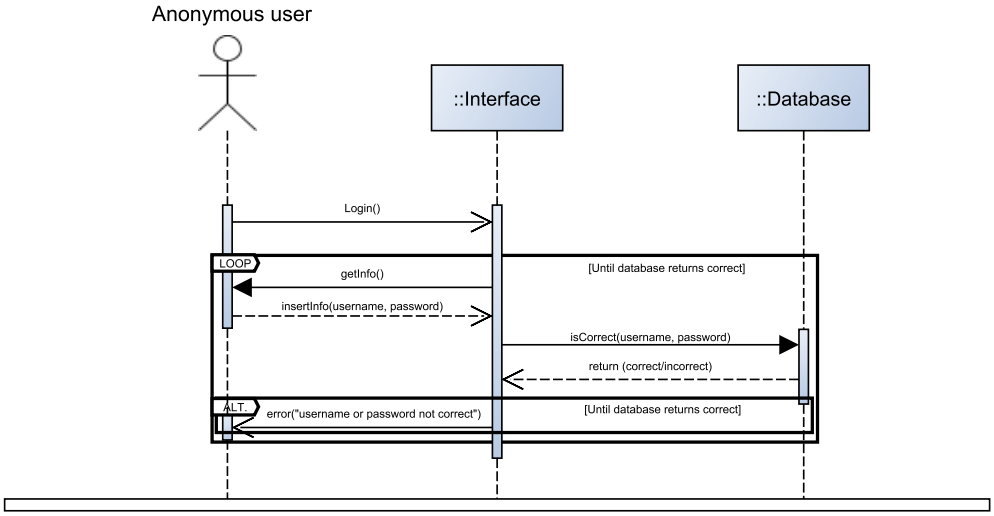
Login

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Login	
Release (id)	UC6, v0.1, 20/04/2026	
Context of Use	User authentication.	
Scope	Let an anonymous user login. FR7 NFR1	
Level	Primary task	
Primary actor	Anonymous user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous user	Wants to login
	System	Manages the user interface, the database and their interactions
Preconditions	The user must be logged out.	
Minimal Guarantees	No user login in an account they have no authorization for.	
Success Guarantees	The anonymous user is logged in its account.	
Trigger	The anonymous user selects “login”.	
Description	**Step**	**Action**
	1	The anonymous user selects the functionality “login”.
	2	The system asks the user to insert username and password.
	3	The user inserts the required data.
	4	The system checks whether both the username and password correspond to an existing account.
	5	The system switches to the authenticated user functionalities.
Extension	**Step**	**Other action**
	4a	Invalid username or password: UC6.1
Technology & data Variations	None	

Alternative flow	Login: Invalid username or password	
Release (id)	UC6.1, v0.2, 25/04/2026	
Context of Use	User authentication.	
Scope	The system alerts the anonymous user that they have inserted an invalid username or password. FR7 NFR1	
Level	Subfunction	
Primary actor	Anonymous user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous user	Wants to login
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted a pair of username and password that does not correspond to any existing account	
Minimal Guarantees	No user login in an account they have no authorization for.	
Success Guarantees	None	
Trigger	UC6	
Description (Main Success Scenario)	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid username or password, specifying the problem.
	2	The sequence of events starts at step 2 of the main flow (UC6).
Technology & data Variations	None	


UC6: Login
Anonymous user
System
Select "login"
Ask username and password
Insert username and password
Incorrect
Check data
Correct
Switch to authenticated user

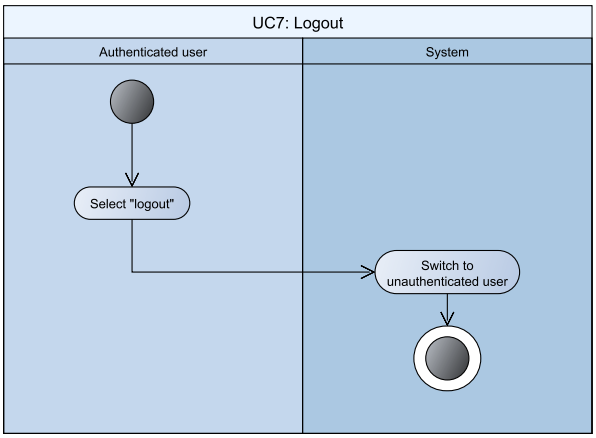

Anonymous user
::Interface
::Database
Login()
LOOP
[Until database returns correct]
getInfo()
insertInfo(username, password)
isCorrect(username, password)
return (correct/incorrect)
ALT.
error("username or password not correct")
[Until database returns correct]

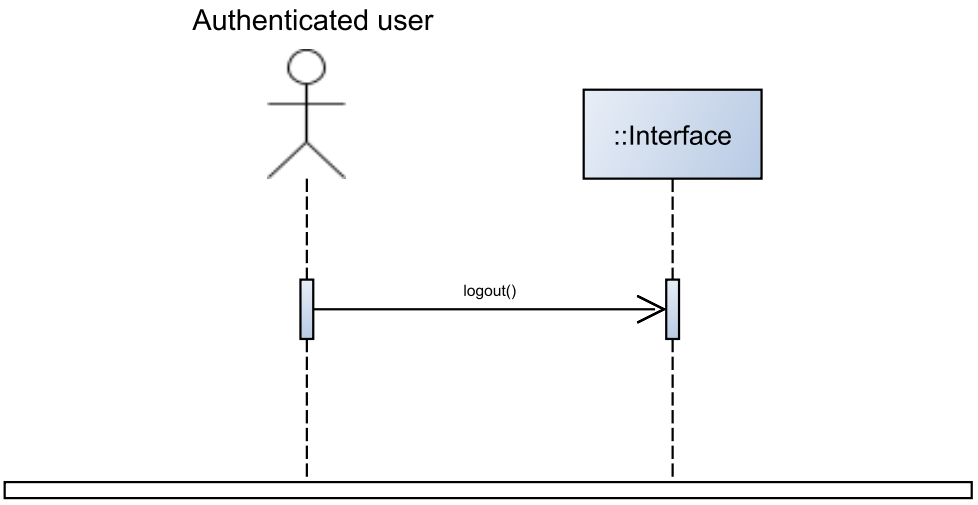
UC6: Login

Logout

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

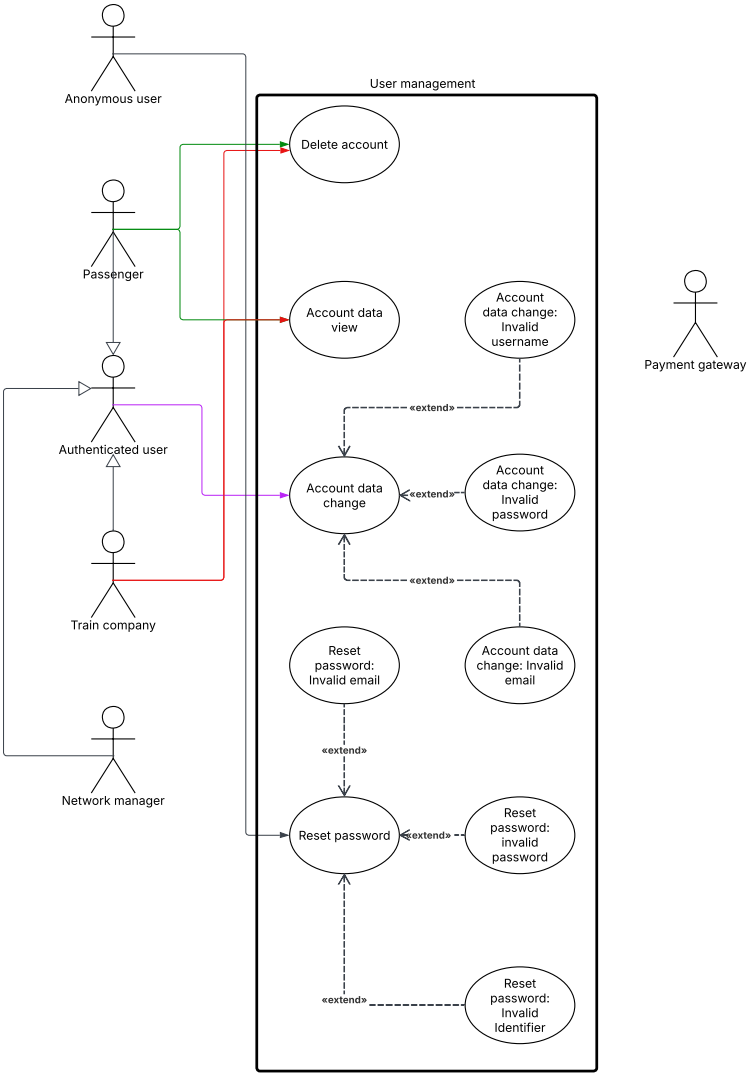
Use case	Logout	
Release (id)	UC7, v0.1, 20/04/2026	
Context of Use	User authentication.	
Scope	Let an authenticated user logout. FR8 NFR1	
Level	Primary task	
Primary actor	Authenticated user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Authenticated user	Wants to logout
	System	Manages the user interface, the database and their interactions
Preconditions	The user must be logged in.	
Minimal Guarantees	None	
Success Guarantees	The authenticated user is logged out.	
Trigger	The authenticated user selects “logout”.	
Description	**Step**	**Action**
	1	The authenticated user selects the functionality “logout”.
	5	The system switches to the anonymous user functionalities.
Extension	**Step**	**Other action**
	-	None
Technology & data Variations	None	


UC7: Logout
Authenticated user
System
Select "logout"
Switch to unauthenticated user


Authenticated user
::Interface
logout()

UC7: Logout

User management

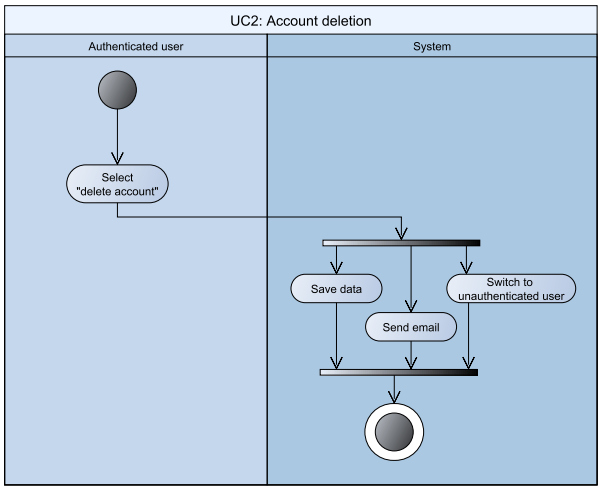


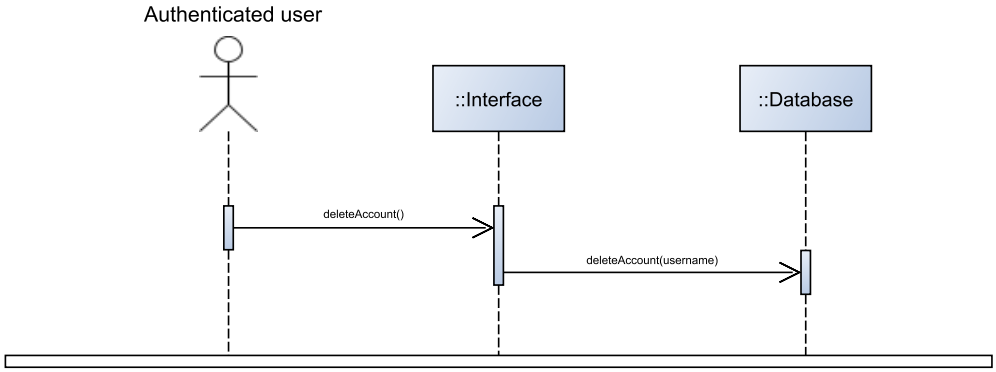
Original image can be found on our Github here

Account deletion

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Account deletion	
Release (id)	UC2, v0.1, 19/04/2026	
Context of Use	Account management.	
Scope	Let an authenticated user delete its account. FR3 NFR1, NFR8, NFR9	
Level	Primary task	
Primary actor	Passenger or train company	
Stakeholder & interest	**Stakeholder**	**Interest**
	Passenger	Wants to delete its account
	Train company	Wants to delete its account
	System	Manages the user interface, the database and their interactions
Preconditions	The user must be logged in.	
Minimal Guarantees	None	
Success Guarantees	The user successfully deleted its account and is logged out.	
Trigger	The user selects “delete account”.	
Description	**Step**	**Action**
	1	The anonymous user selects the functionality “delete account”.
	2	The system notifies the user of successful deletion with an email.
	3	The system removes the data in the account archive.
	4	The system switches to the unauthenticated (anonymous) user functionalities.
Extension	**Step**	**Other action**
	-	-
Technology & data Variations	None	



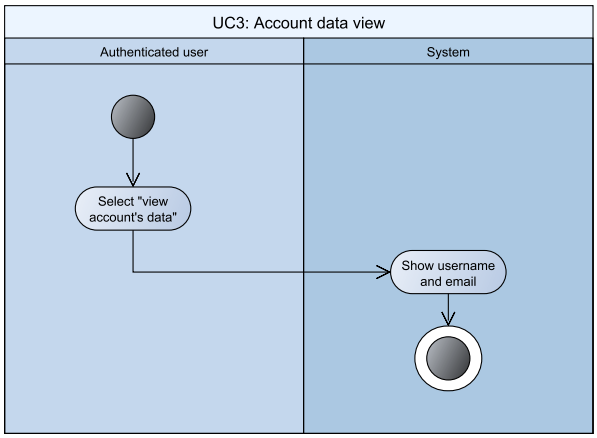


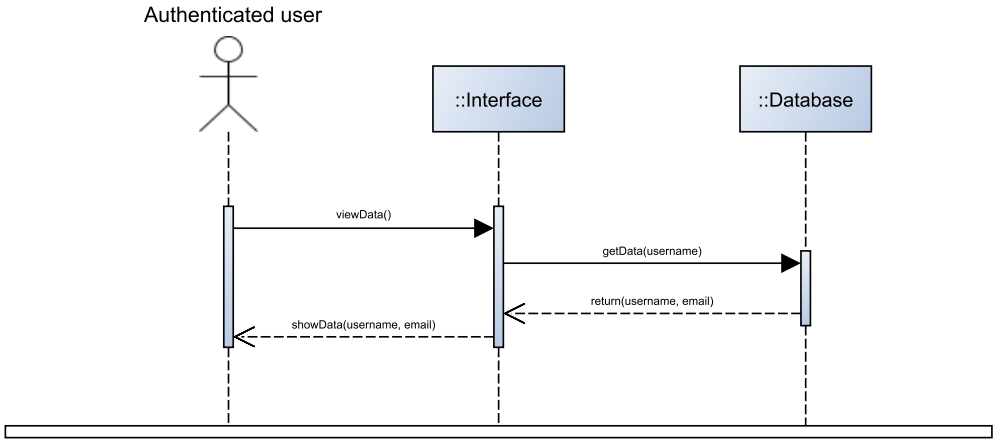
UC2: Account deletion

Account data view

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Account data view	
Release (id)	UC3, v0.2, 25/04/2026	
Context of Use	Account management.	
Scope	Let an authenticated user view their account's data. FR4 NFR1, NFR7	
Level	Primary task	
Primary actor	Authenticated user (passenger, train company)	
Stakeholder & interest	**Stakeholder**	**Interest**
	Authenticated user	Wants to view its account's data
	System	Manages the user interface, the database and their interactions
Preconditions	The user must be logged in.	
Minimal Guarantees	None	
Success Guarantees	The user successfully views its account's data.	
Trigger	The anonymous user selects "view account's data".	
Description	**Step**	**Action**
	1	The anonymous user selects the functionality "view account's data".
	2	The system shows the data saved in the account archive, that is username and email.
Extension	**Step**	**Other action**
	-	-
Technology & data Variations	None	





UC3: Account data view

Account data change

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Account data change		
Release (id)	UC4, v0.2, 25/04/2026		
Context of Use	Account management.		
Scope	Let an authenticated user change its account's data. FR5 NFR1, NFR2, NFR3, NFR4, NFR5, NFR8, NFR9		
Level	Primary task		
Primary actor	Authenticated user (passenger, train company)		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Authenticated user	Wants to change its account's data	
	System	Manages the user interface, the database and their interactions	
Preconditions	The user must be logged in.		
Minimal Guarantees	No duplicated or unsecure account is created with a modification.		
Success Guarantees	The authenticated user successfully changed its account's data.		
Trigger	The authenticated user selects "change data".		
Description	**Step**	**Action**	
	1	The authenticated user selects the functionality "change data".	
	2	The system asks the user which data to modify.	
	3	If the user selects username	
	3.1	The system asks the user to insert the new username.	
	3.2	The user inserts the new username.	
	3.3	The system checks whether an account with the same username already exists.	
	4	If the user selects email	
	4.1	The system asks the user to insert the new email.	
	4.2	The user inserts the new email.	
	4.3	The system checks whether an account with the same email already exists.	

	5	If the user selects password
	5.1	The system asks the user to insert the password twice.
	5.2	The user inserts the required data.
	5.3	The system checks whether the password satisfies the security requirements and if the two passwords coincide.
	6	The system saves the data in the account archive.
	7	The system notifies the user of successful data change with an email.
Extension	**Step**	**Other action**
	3.3a	Invalid username or email: UC4.1
	4.3a	Invalid email: UC4.2
	5.3a	Invalid password: UC4.3
Technology & data Variations	None	

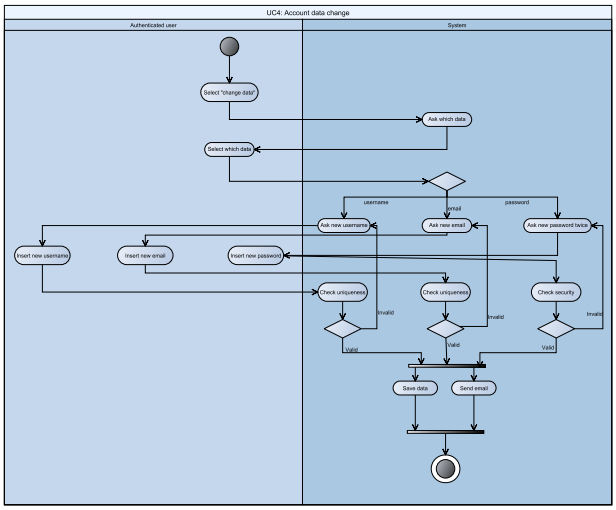
Alternative flow	Account data change: Invalid username	
Release (id)	UC4.1, v0.2, 25/04/2026	
Context of Use	Account management.	
Scope	The system alerts the authenticated user that they have inserted an invalid username. FR5 NFR1, NFR2, NFR3	
Level	Subfunction	
Primary actor	Authenticated user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Authenticated user	Wants to change username
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted an already existing username.	
Minimal Guarantees	No duplicated account is created.	

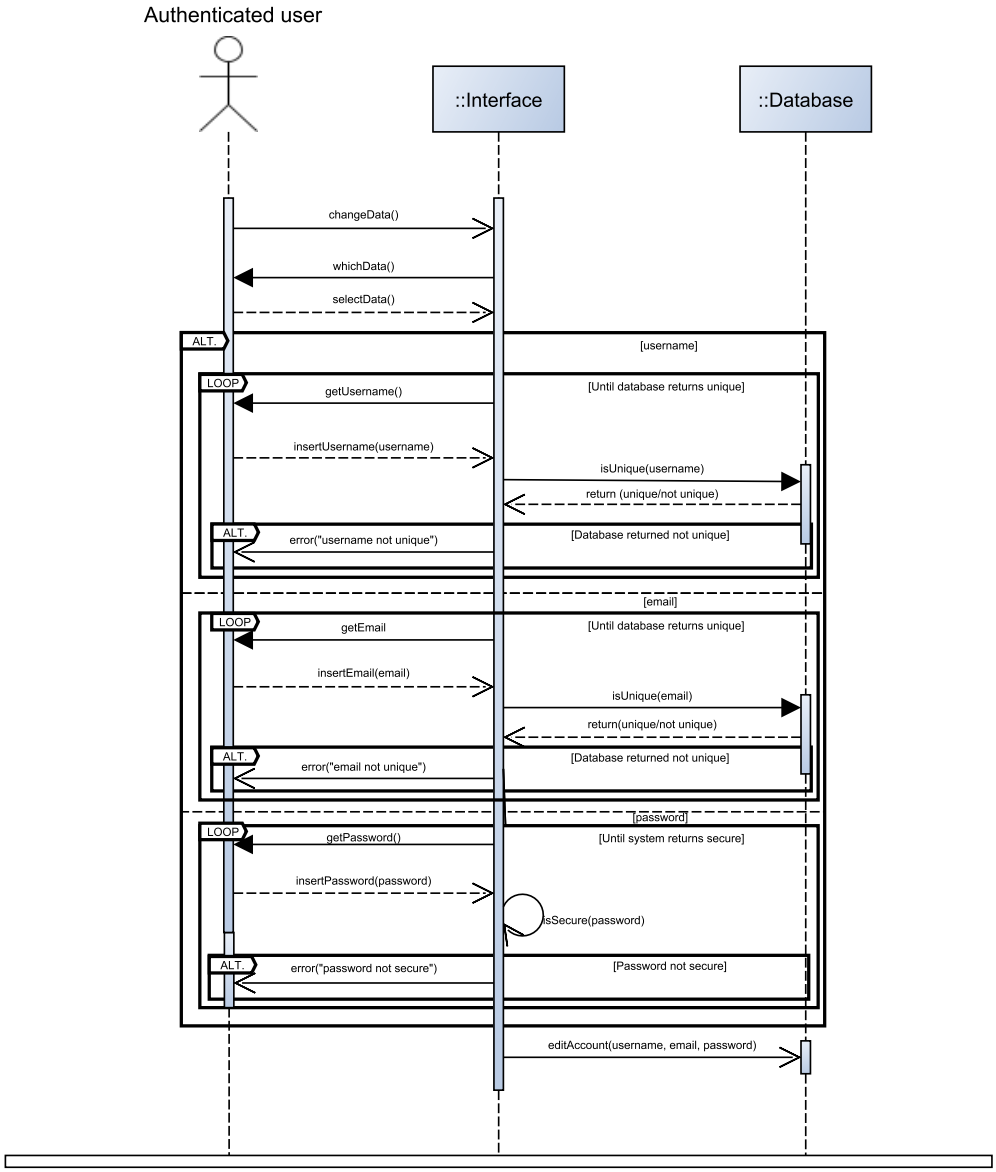
Success Guarantees	None	
Trigger	UC4	
Description	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid username.
	2	The sequence of events starts at step 3.1 of the main flow (UC4).
Technology & data Variations	None	

Alternative flow	Account data change: Invalid email	
Release (id)	UC4.2, v0.2, 25/04/2026	
Context of Use	Account management.	
Scope	The system alerts the authenticated user that they have inserted an invalid email. FR5 NFR1, NFR2, NFR3	
Level	Subfunction	
Primary actor	Authenticated user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Authenticated user	Wants to change email
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted an already existing email.	
Minimal Guarantees	No duplicated account is created.	
Success Guarantees	None	
Trigger	UC4	
Description	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid email.

	2	The sequence of events starts at step 4.1 of the main flow (UC4).
Technology & data Variations	None	

Alternative flow	Account data change: Invalid password	
Release (id)	UC4.3, v0.2, 25/04/2026	
Context of Use	Account management.	
Scope	The system alerts the authenticated user that they have inserted an invalid password. FR5 NFR1, NFR2, NFR4	
Level	Subfunction	
Primary actor	Authenticated user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Authenticated user	Wants to change password
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted a password that does not satisfy the security requirements or the two passwords do not coincide.	
Minimal Guarantees	No unsecure account is created.	
Success Guarantees	None	
Trigger	UC4	
Description (Main Success Scenario)	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid password, specifying the problem.
	2	The sequence of events starts at step 5.1 of the main flow (UC4).
Technology & data Variations	None	


UC4: Account data change
Authenticated user
System
Select "change data"
Ask which data
Select which data
username
email
password
Ask new username
Ask new email
Ask new password twice
Insert new username
Insert new email
Insert new password
Check uniqueness
Check uniqueness
Check security
Invalid
Invalid
Invalid
Valid
Valid
Valid
Save data
Send email



UC4: Account data change

Reset password

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

<table>
<tr><td>Use case</td><td colspan="3">Reset password</td></tr>
<tr><td>Release (id)</td><td colspan="3">UC5, v0.1, 25/04/2026</td></tr>
<tr><td>Context of Use</td><td colspan="3">Account management.</td></tr>
<tr><td>Scope</td><td colspan="3">Let a user reset the password.
FR6
NFR1, NFR2, NFR4, NFR5, NFR8</td></tr>
<tr><td>Level</td><td colspan="3">Primary task</td></tr>
<tr><td>Primary actor</td><td colspan="3">Anonymous user</td></tr>
<tr><td rowspan="3">Stakeholder & interest</td><td colspan="2">Stakeholder</td><td>Interest</td></tr>
<tr><td colspan="2">Anonymous user</td><td>Wants to reset the password</td></tr>
<tr><td colspan="2">System</td><td>Manages the user interface, the database and their interactions</td></tr>
<tr><td>Preconditions</td><td colspan="3">The user must be logged out.</td></tr>
<tr><td>Minimal Guarantees</td><td colspan="3">No unsecure account is created with a modification.
No user can change the password of another user.</td></tr>
<tr><td>Success Guarantees</td><td colspan="3">The user successfully changed the password.</td></tr>
<tr><td>Trigger</td><td colspan="3">The user selects “forgot password”.</td></tr>
<tr><td rowspan="11">Description</td><td>Step</td><td colspan="2">Action</td></tr>
<tr><td>1</td><td colspan="2">The user selects the functionality “forgot password”.</td></tr>
<tr><td>2</td><td colspan="2">The system asks the user to insert the email associated to its account.</td></tr>
<tr><td>3</td><td colspan="2">The user inserts the email.</td></tr>
<tr><td>4</td><td colspan="2">The system checks if the email is associated to an existing account.</td></tr>
<tr><td>5</td><td colspan="2">The system creates a random identifier.</td></tr>
<tr><td>6</td><td colspan="2">The system sends the identifier to the email.</td></tr>
<tr><td>7</td><td colspan="2">The system asks the user to insert the identifier it sent.</td></tr>
<tr><td>8</td><td colspan="2">The user inserts the identifier.</td></tr>
<tr><td>9</td><td colspan="2">The system checks whether the identifier is correct.</td></tr>
<tr><td>10</td><td colspan="2">The system asks the user to insert the password twice.</td></tr>
</table>

	11	The user inserts the required data.
	12	The system checks whether the password satisfies the security requirements and if the two passwords coincide.
	13	The system saves the data in the account archive.
	14	The system notifies the user of successful password change with an email.
Extension	**Step**	**Other action**
	4a	Invalid email: UC5.1
	9a	Invalid identifier: UC5.2
	12a	Invalid password: UC5.3
Technology & data Variations	None	

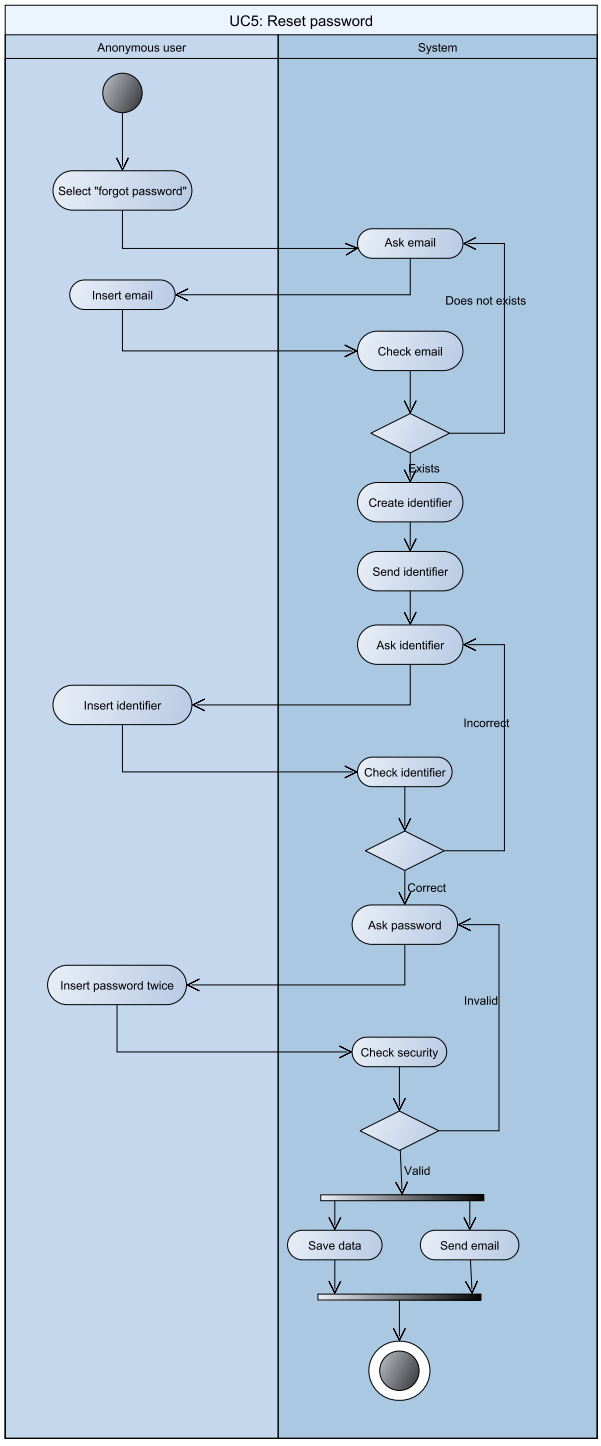
Alternative flow	Reset password: Invalid email	
Release (id)	UC5.1, v0.1, 25/04/2026	
Context of Use	Account management.	
Scope	The system alerts the authenticated user that they have inserted an invalid email. FR6 NFR1	
Level	Subfunction	
Primary actor	Anonymous or Authenticated user (Passenger, Train company or Network Manager)	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous/Authenticated user	Wants to reset password
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted an email that does not correspond to any existing account.	
Minimal Guarantees	None	
Success Guarantees	None	

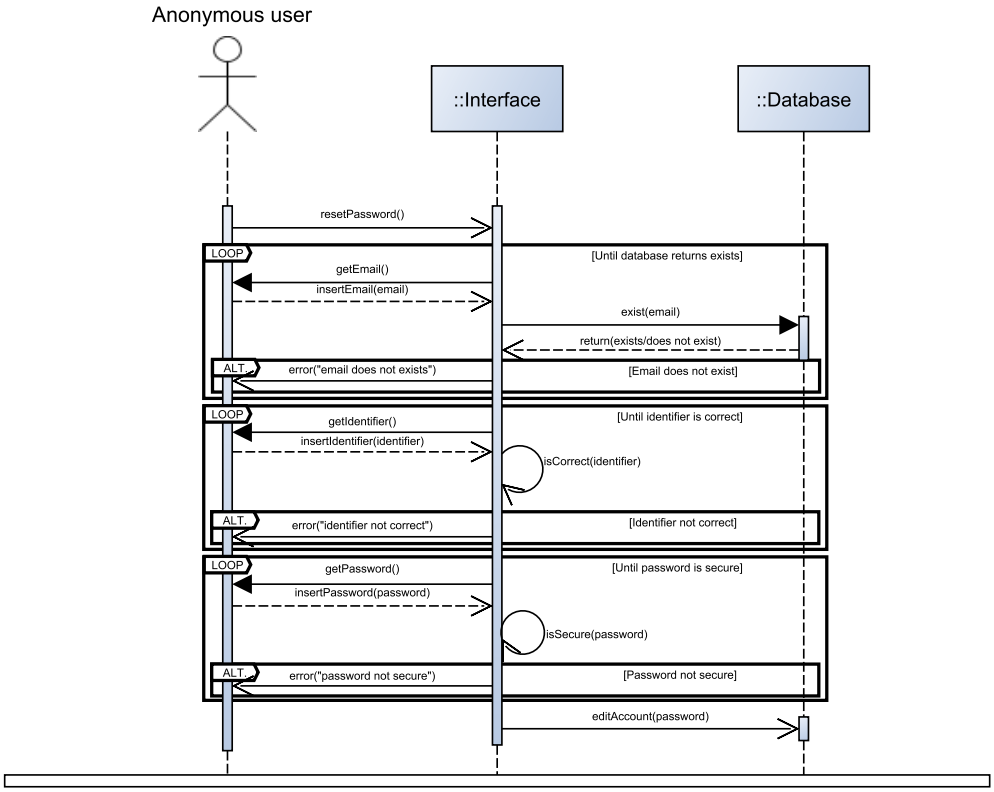
Trigger	UC5	
Description	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid email.
	2	The sequence of events starts at step 2a.1 of the main flow (UC5).
Technology & data Variations	None	

Alternative flow	Reset password: Invalid identifier	
Release (id)	UC5.2, v0.1, 25/04/2026	
Context of Use	Account management.	
Scope	The system alerts the authenticated user that they have inserted an invalid identifier. FR6 NFR1	
Level	Subfunction	
Primary actor	Anonymous or Authenticated user (Passenger, Train Company or Network Manager)	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous/Authenticated user	Wants to reset password
	System	Manages the user interface, the database and their interactions
Preconditions	The user inserted an identifier that does not correspond to the one created by the system.	
Minimal Guarantees	None	
Success Guarantees	None	
Trigger	UC5	
Description	**Step**	**Action**
	1	The system alerts the user that has inserted an invalid identifier.

	2	The sequence of events starts at step 5 of the main flow (UC5).
Technology & data Variations	None	

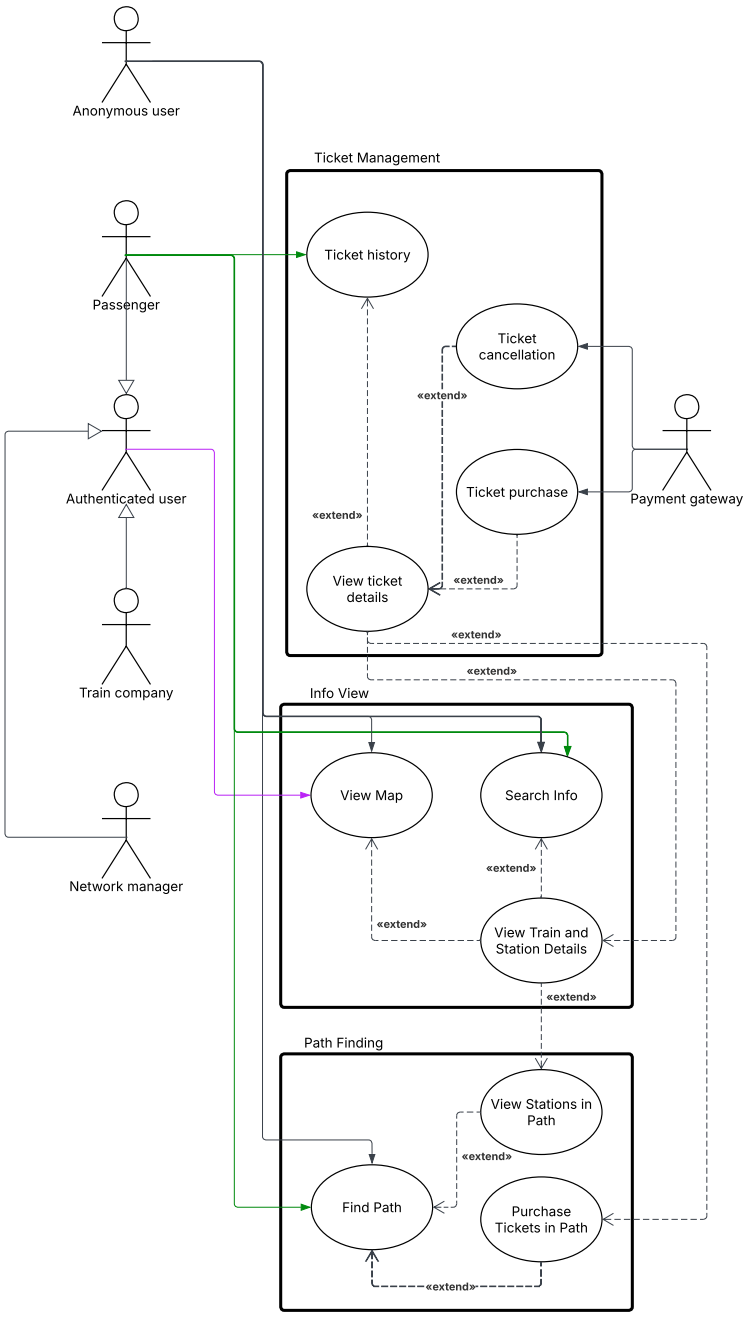
Alternative flow	Reset password: Invalid password		
Release (id)	UC5.3, v0.1, 25/04/2026		
Context of Use	Account management.		
Scope	The system alerts the anonymous user that they have inserted an invalid password. FR6 NFR1, NFR2, NFR4		
Level	Subfunction		
Primary actor	Anonymous or Authenticated user (Passenger, Train Company or Network Manager)		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Anonymous/Authenticated user	Wants to reset password	
	System	Manages the user interface, the database and their interactions	
Preconditions	The user inserted a password that does not satisfy the security requirements or the two passwords do not coincide.		
Minimal Guarantees	No unsecure password is created.		
Success Guarantees	None		
Trigger	UC5		
Description (Main Success Scenario)	**Step**	**Action**	
	1	The system alerts the user that has inserted an invalid password, specifying the problem.	
	2	The sequence of events starts at step 8 of the main flow (UC5).	
Technology & data Variations	None		


UC5: Reset password
Anonymous user
System
Select "forgot password"
Ask email
Insert email
Does not exists
Check email
Exists
Create identifier
Send identifier
Ask identifier
Insert identifier
Incorrect
Check identifier
Correct
Ask password
Insert password twice
Invalid
Check security
Valid
Save data
Send email



UC5: Reset password

Ticketing management

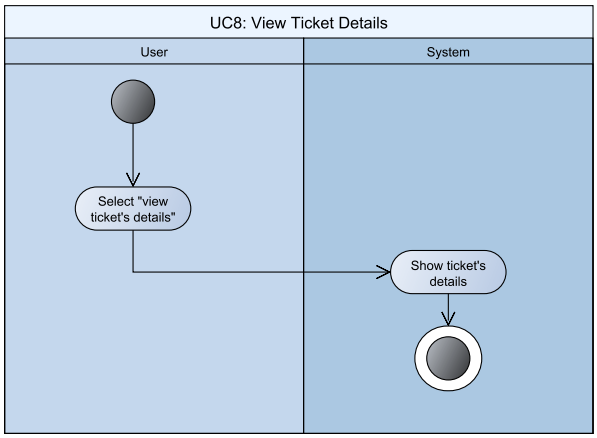


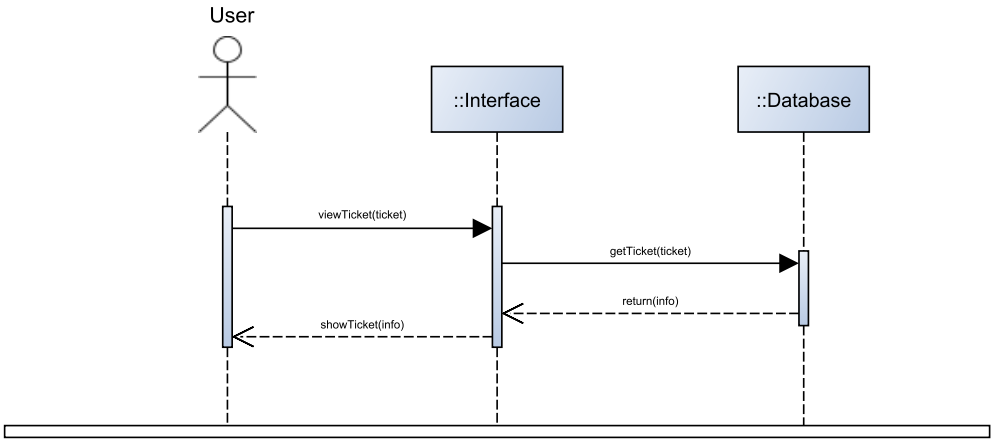
Original image can be found on our Github here

View Ticket Details

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	View Ticket Details	
Release (id)	UC8, v0.1, 21/04/2026	
Context of Use	Ticketing management.	
Scope	Let a user view the details of a ticket. FR9	
Level	Primary task	
Primary actor	Passenger	
Stakeholder & interest	**Stakeholder**	**Interest**
	Passenger	Wants to view tickets' details
	Anonymous user	Wants to view tickets' details
	System	Manages the user interface, the database and their interactions
Preconditions	The user must be in a use case where a ticket is displayed (UC11, UC12.1).	
Minimal Guarantees	None	
Success Guarantees	The user views the ticket's details.	
Trigger	The user selects "view tickets' details".	
Description	**Step**	**Action**
	1	The user selects the functionality "view tickets' details".
	2	The system displays the ticket's details.
Extension	**Step**	**Other action**
	2a	The authenticated Passenger can buy the not yet owned ticket (UC9 Ticket Purchase)
	2b	The authenticated Passenger can cancel the bought ticket (UC10 Ticket Cancellation)
Technology & data Variations	None	





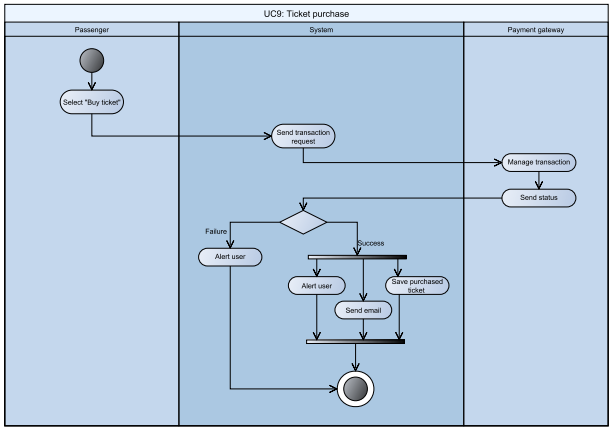
UC8: View Ticket Details

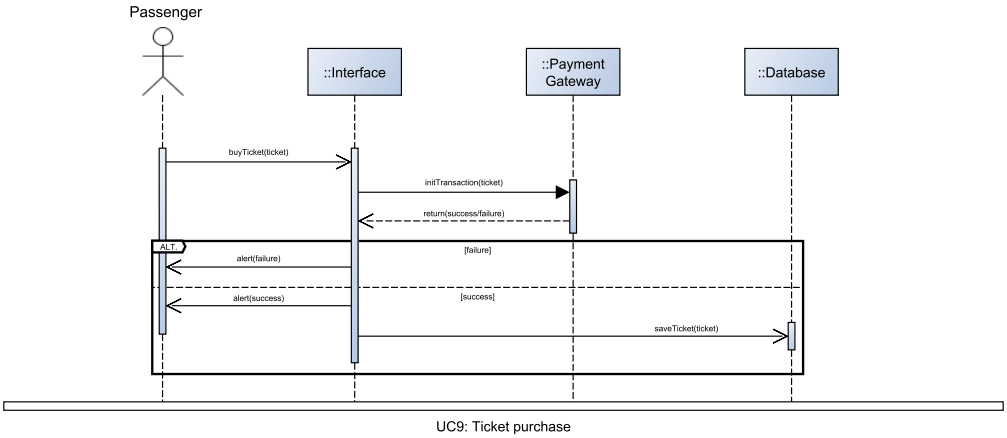
Ticket purchase

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Ticket purchase	
Release (id)	UC9, v0.2, 25/04/2026	
Context of Use	Ticketing management.	
Scope	Enable a passenger to purchase a ticket. FR10	
Level	Primary task	
Primary actor	Passenger	
Stakeholder & interest	**Stakeholder**	**Interest**
	Passenger	Wants to buy a ticket
	Payment gateway	Manages transaction
	System	Manages the user interface, the database and their interactions
Preconditions	The passenger must be logged in. The passenger must be viewing the ticket's details (UC8 View Ticket Details).	
Minimal Guarantees	The passenger who cannot pay does not purchase any ticket.	
Success Guarantees	The passenger successfully purchased a ticket.	
Trigger	The passenger selects "Buy ticket" from UC View Ticket Details.	
Description	**Step**	**Action**
	1	The passenger selects the functionality "Buy ticket".
	2	The system sends a transaction request to the payment gateway.
	3	The payment gateway manages the transaction.
	4	The payment gateway sends the transaction's status to the system.
	5	If the transaction is not successful
	5.1	The system alerts the passenger about the failed transaction.
	6	Otherwise

	6.1	The system alerts the passenger about the completed transaction.
	6.2	The system notifies the passenger of successful purchase with an email.
	6.3	The system saves the ticket in the account archive.
Extension	**Step**	**Other action**
	-	None
Technology & data Variations	All operations are analogous to Group Pass, where a passenger buy n tickets	





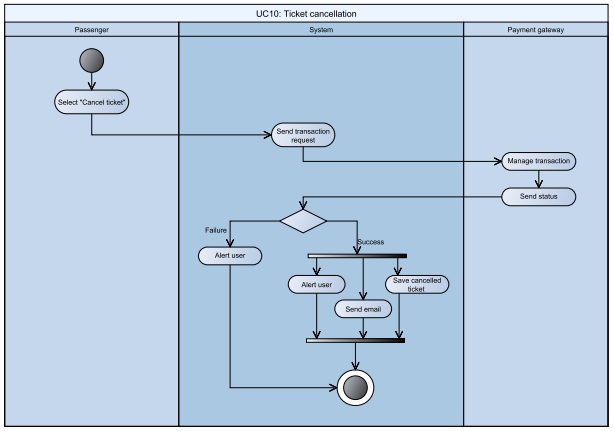
UC9: Ticket purchase

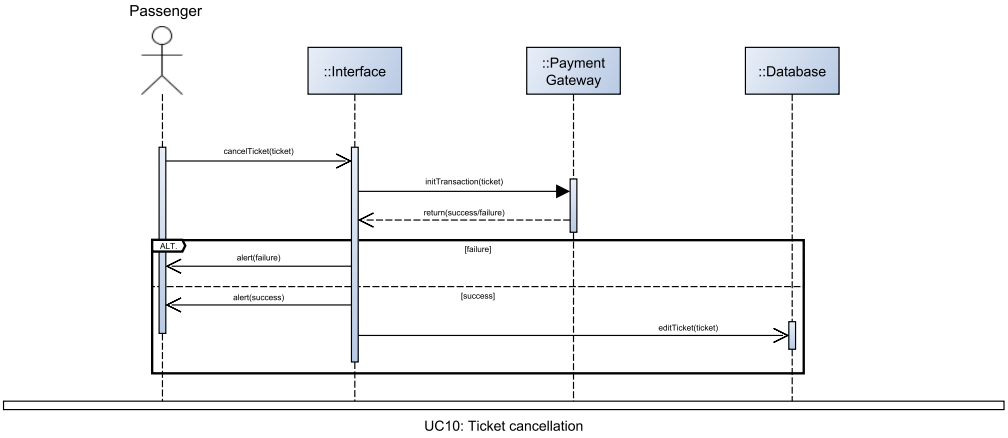
Ticket cancellation

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Ticket cancellation	
Release (id)	UC10, v0.1, 21/04/2026	
Context of Use	Ticketing management.	
Scope	Enable a passenger to cancel a purchased ticket. FR11	
Level	Primary task	
Primary actor	Passenger	
Stakeholder & interest	**Stakeholder**	**Interest**
	Passenger	Wants to buy a ticket
	Payment gateway	Manages transaction
	System	Manages the user interface, the database and their interactions
Preconditions	The passenger must be logged in. The passenger must be viewing the ticket's details (UC8). The passenger must have already purchased the ticket.	
Minimal Guarantees	The passenger does not cancel any ticket they have not purchased.	
Success Guarantees	The passenger successfully cancelled a purchased ticket.	
Trigger	The passenger selects "Cancel ticket".	
Description	**Step**	**Action**
	1	The passenger selects the functionality "Cancel ticket".
	2	The system sends a transaction request to the payment gateway.
	3	The payment gateway manages the transaction.
	4	The payment gateway sends the transaction's status to the system.
	5	If the transaction is not successful
	5.1	The system alerts the passenger about the failed transaction.
	6	Otherwise

	6.1	The system alerts the passenger about the completed transaction.
	6.2	The system notifies the passenger of successful purchase with an email.
	6.3	The system saves the ticket's cancellation information in the account archive.
Extension	**Step**	**Other action**
	-	None
Technology & data Variations	None	



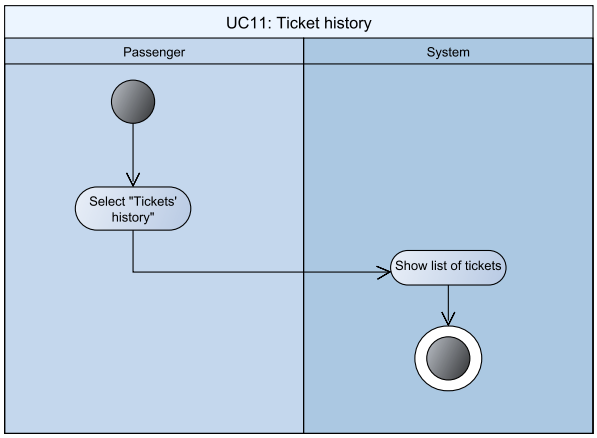


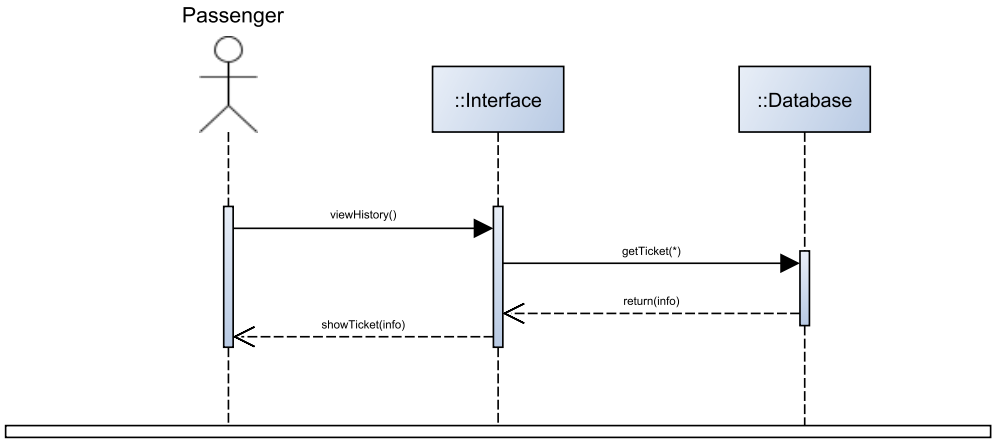
UC10: Ticket cancellation

Ticket history

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

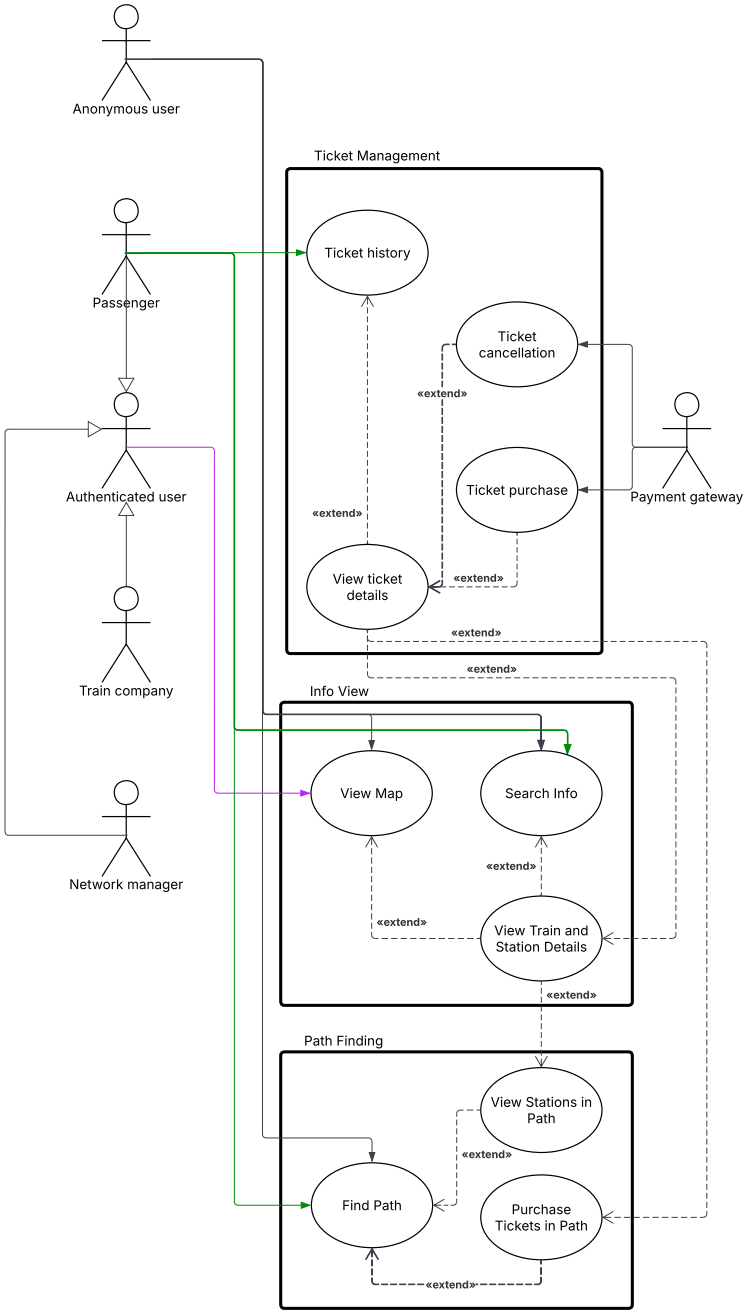
Use case	Ticket history	
Release (id)	UC11, v0.1, 21/04/2026	
Context of Use	Ticketing management.	
Scope	Enable a passenger to view its tickets' history, both purchased and cancelled. FR12	
Level	Primary task	
Primary actor	Passenger	
Stakeholder & interest	**Stakeholder**	**Interest**
	Passenger	Wants to view tickets' history
	System	Manages the user interface, the database and their interactions
Preconditions	The passenger must be logged in.	
Minimal Guarantees	None	
Success Guarantees	The passenger successfully view tickets' history.	
Trigger	The passenger selects "Tickets' history".	
Description	**Step**	**Action**
	1	The passenger selects the functionality "Tickets' history".
	2	The system shows the list of tickets purchased and cancelled by that passenger.
Extension	**Step**	**Other action**
	2a	The passenger can select a ticket to view its details (UC8)
Technology & data Variations	None	





UC11: Ticket history

Path finding



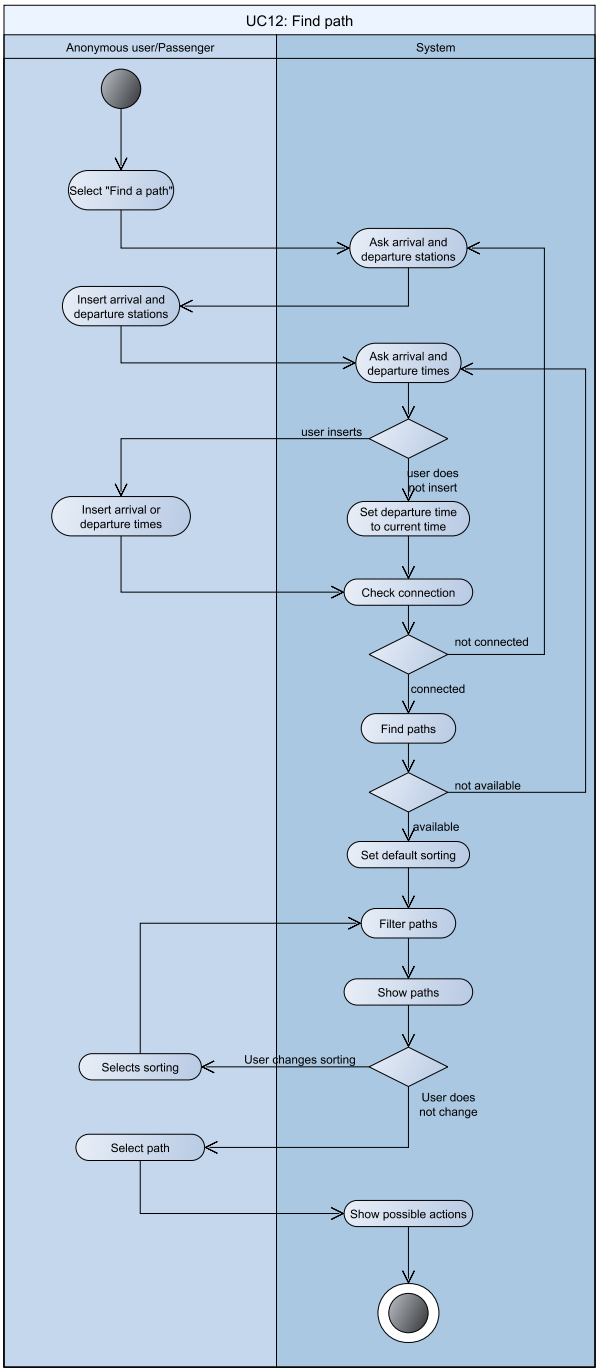
Original image can be found on our Github here

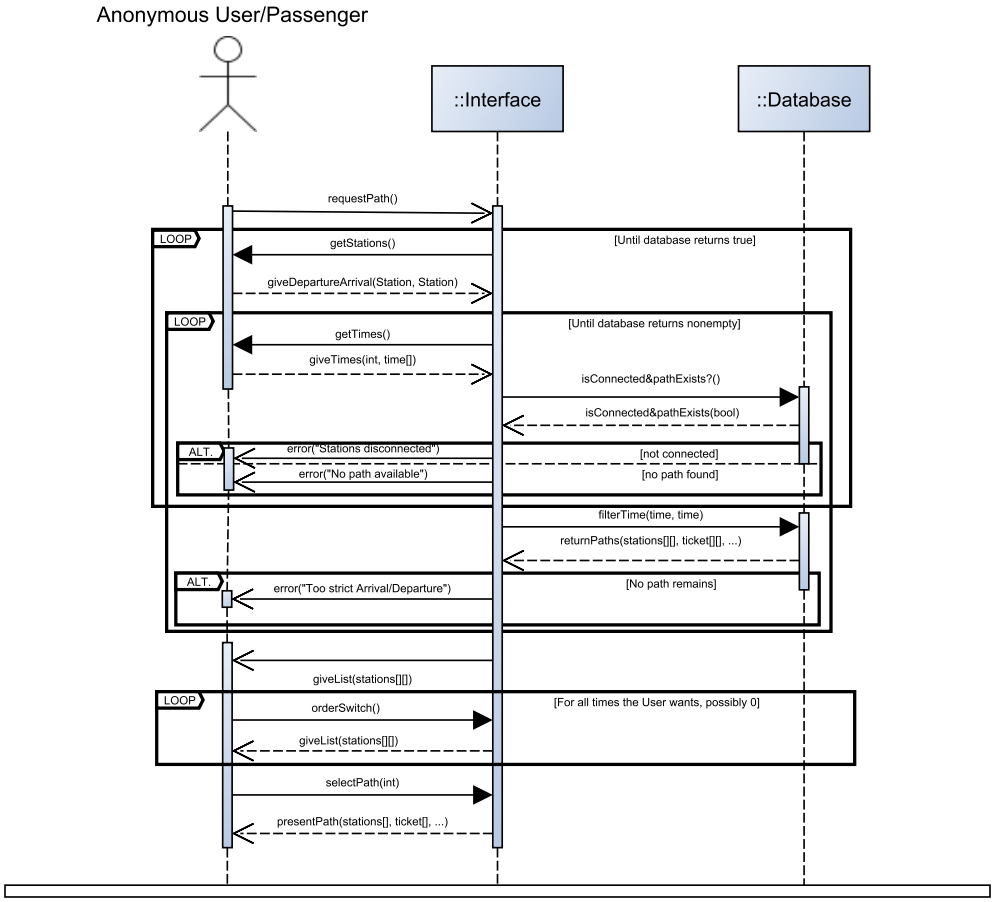
Find Path

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Find Path		
Release (id)	UC12, v0.1, 25/04/2026		
Context of Use	Path finding		
Scope	Let users find a path between two stations. FR13, FR14, FR15, FR16 NFR12, NFR13		
Level	Primary task		
Primary actor	Anonymous user OR Passenger		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Anonymous user or Passenger	Find a path	
	System	Manages the user interface, the database and their interactions	
Preconditions	The user must be either logged out or logged in as a Passenger		
Minimal Guarantees	No impossible path is proposed		
Success Guarantees	The user has a suitable path and can directly select other operations for it.		
Trigger	The user selects “find a path”.		
Description	**Step**	**Action**	
	1	The user selects the functionality “find a path”.	
	2	The system asks the user to insert a Departing and Arriving station	
	3	The user inserts all the requested data.	
	4	The system asks the user a Departure or Arrival time	
	5	The user provides any of the requested data, possibly confirming zero	
	6	The system checks whether at least one path along the mesh between the two stations exists	
	7	The system computes whether one or the intersection of multiple service lines (referred to as candidate paths) connects the stations	

	8	The system filters out all candidate paths that do not respect Departure and Arrival times, if provided
	9	The system presents the user an option to change path-sorting algorithm from a default option
	10	The system uses the path-sorting algorithm to provide the top-ranking paths computed at step 7
	11	The user selects one path
	12	The system presents to the user all path-specific actions regarding that path (UC 12.1, 12.2)
Extension	**Step**	**Other action**
	5a	If the user inserts zero data
	5a.1	Default to “Departure” as current time, proceed normally
	6a	If no path is found
	6a.1	Produce “Stations disconnected” error and return to step 2
	7a	If no candidate paths are found
	7a.1	Produce “No path available” error and return to step 2
	8a	If no candidate paths remain after the filter
	8a.1	Produce “Too strict Arrival/Departure” error and return to step 4
	9-10a	If the user changes sorting algorithm
	9-10a.1	The newly selected one is used to repeat from step 10
	9-10b	If two or more paths score equally in selected sorting algorithm
	9.10b.1	Use all other sorting algorithms in a pre-determined order as tie-breakers, and failing that, assign order among them randomly. Proceed as normal
Technology & data Variations	None	


UC12: Find path
Anonymous user/Passenger
System
Select "Find a path"
Ask arrival and departure stations
Insert arrival and departure stations
Ask arrival and departure times
user inserts
user does not insert
Insert arrival or departure times
Set departure time to current time
Check connection
not connected
connected
Find paths
not available
available
Set default sorting
Filter paths
Show paths
Selects sorting
User changes sorting
User does not change
Select path
Show possible actions



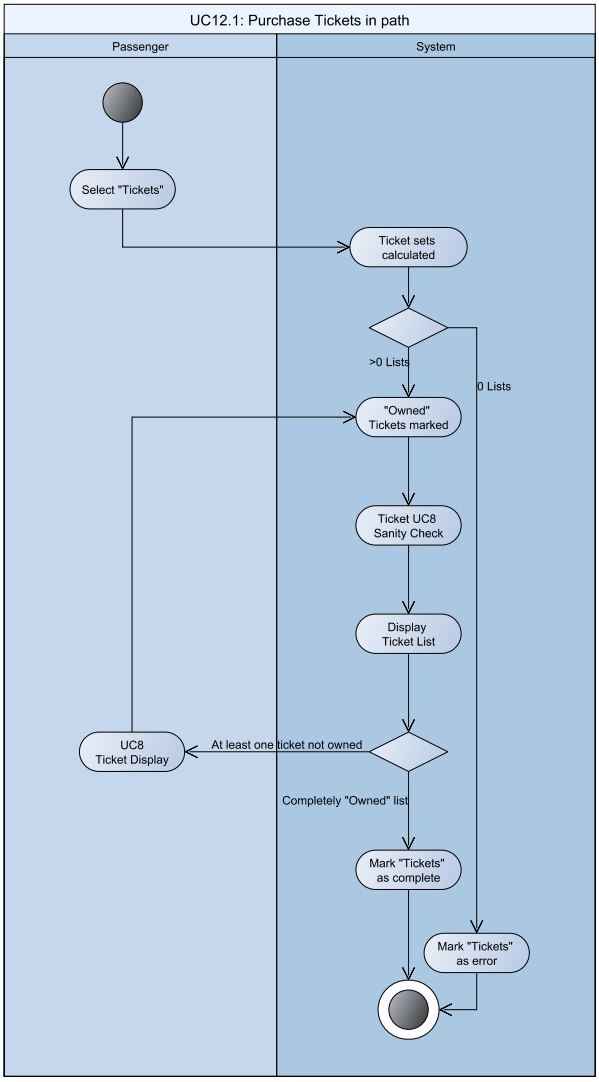
UC12: Find Path

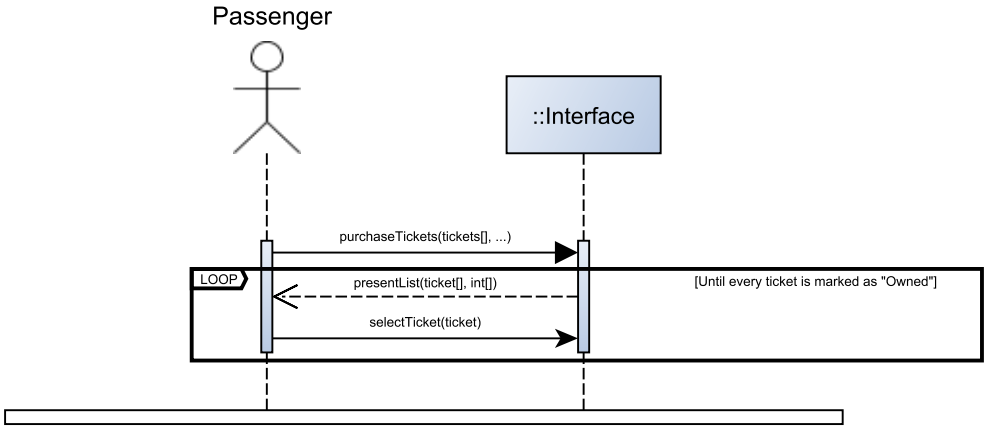
Purchase Tickets in path

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Purchase Tickets in path		
Release (id)	UC12.1, v0.1, 28/04/2026		
Context of Use	Path finding, Ticket Purchase		
Scope	Let a Passenger directly purchase tickets involved in path. FR10		
Level	Subfunction		
Primary actor	Passenger		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Passenger	Purchase tickets	
	System	Manages the user interface, the database and their interactions	
Preconditions	The user must be either logged in as a Passenger		
Minimal Guarantees	No unnecessary tickets are proposed		
Success Guarantees	The user purchased each ticket		
Trigger	The user selects “Tickets” at step 12 of UC12		
Description	**Step**	**Action**	
	1	The Passenger selects “Tickets”	
	2	The system computes the cheapest set of tickets that cover transit in all trains of the path for the duration of the path itself	
	3	All tickets already owned by the passenger are marked as “Owned” on the list.	
	4	The system provides such list to the user, along with the option to view the details (UC8) for each.	
	5	Once a ticket is bought, if not all tickets are marked as “Owned”, return to step 3	
	6	Return to UC12 Step 3, marking “Tickets” option as complete	
Extension	**Step**	**Other action**	
	2a	If no set of available tickets exists	

	2a.1	Produce “Ticketing not available” error, return to UC12 step 13, marking “Tickets” option as error
Technology & data Variations	None	


UC12.1: Purchase Tickets in path
Passenger
System
Select "Tickets"
Ticket sets calculated
>0 Lists
0 Lists
"Owned" Tickets marked
Ticket UC8 Sanity Check
Display Ticket List
UC8 Ticket Display
At least one ticket not owned
Completely "Owned" list
Mark "Tickets" as complete
Mark "Tickets" as error

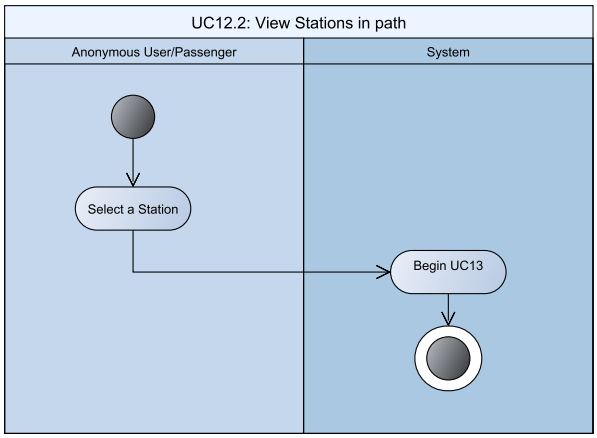


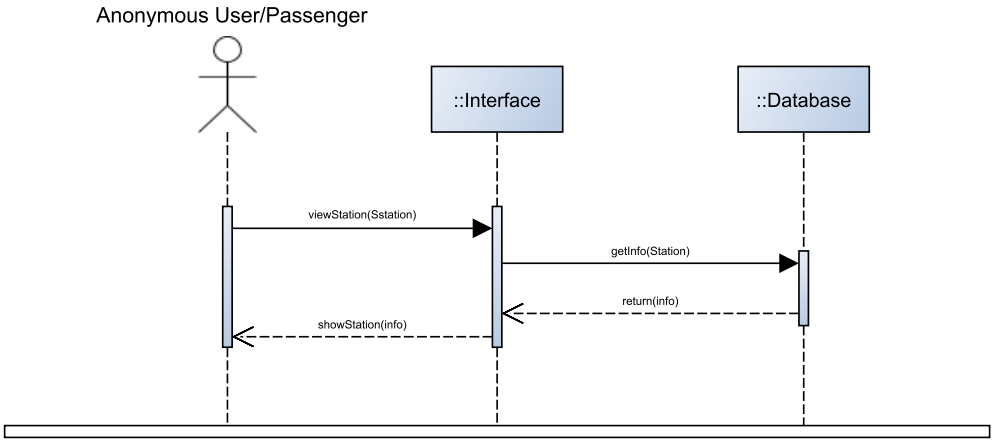
UC12.1: Purchase Tickets in Path

View Stations in path

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

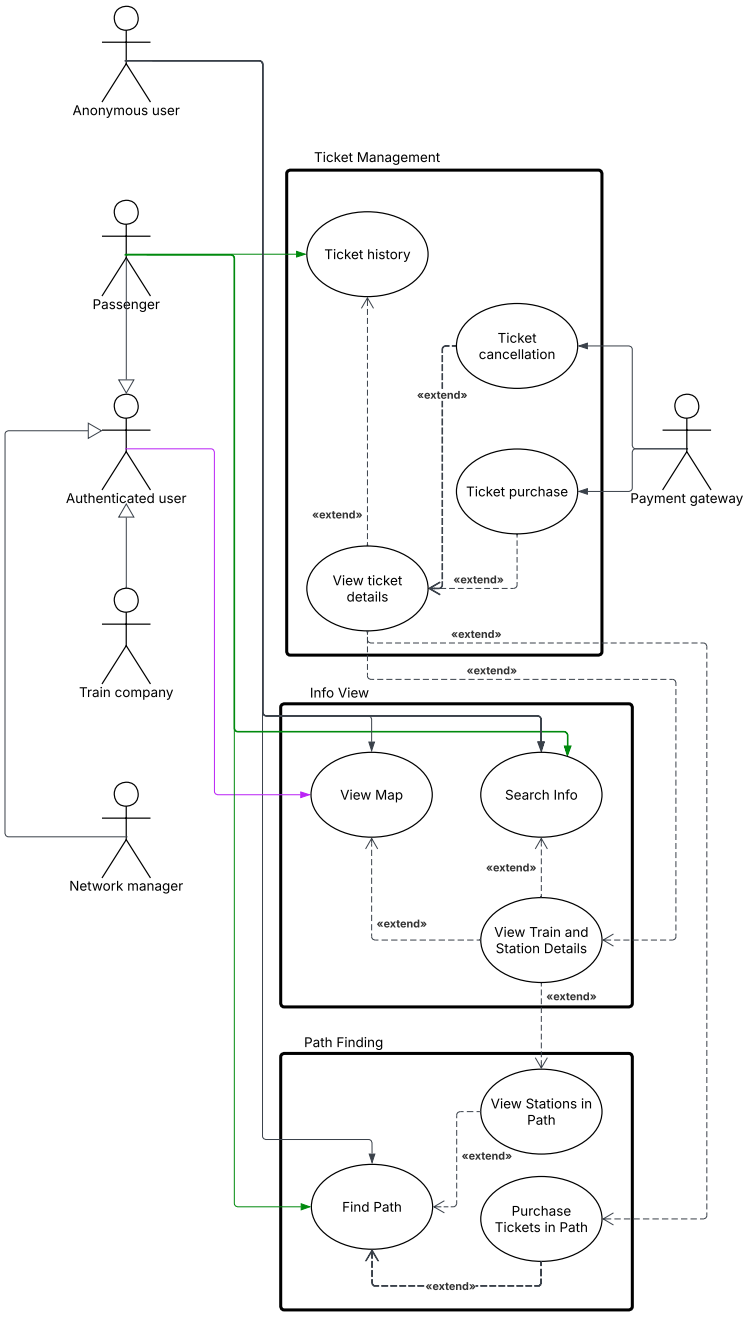
Use case	View Stations in path	
Release (id)	UC12.2, v0.1, 28/04/2026	
Context of Use	Path Finding	
Scope	Let users directly transition to viewing a station's information from a path FR17 NFR11	
Level	Subfunction	
Primary actor	Anonymous User or Passenger	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous User/Passenger	View information
	System	Manages the user interface, the database and their interactions
Preconditions	The path must include at least one station	
Minimal Guarantees	None	
Success Guarantees	The user enters UC13 for a Station	
Trigger	The user selects a station in the path at step 12 of UC12	
Description	**Step**	**Action**
	1	The user selects a station
	2	The system initiates UC13
Extension	**Step**	**Other action**
		None
Technology & data Variations	None	





UC12.2: View stations in Path

Info view



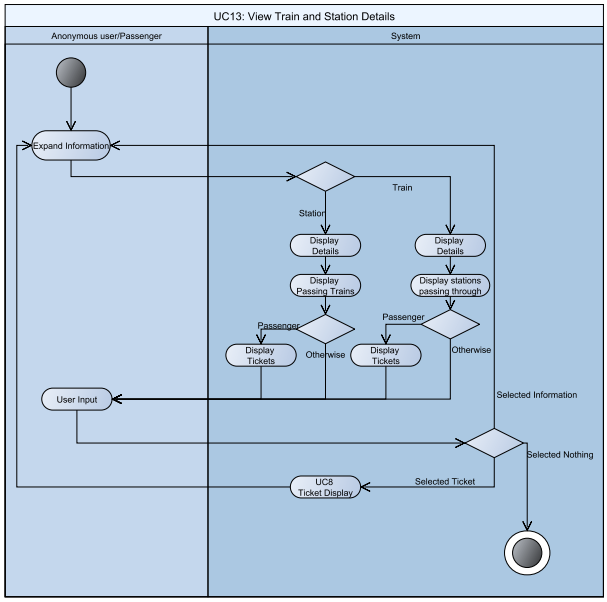
Original image can be found on our Github here

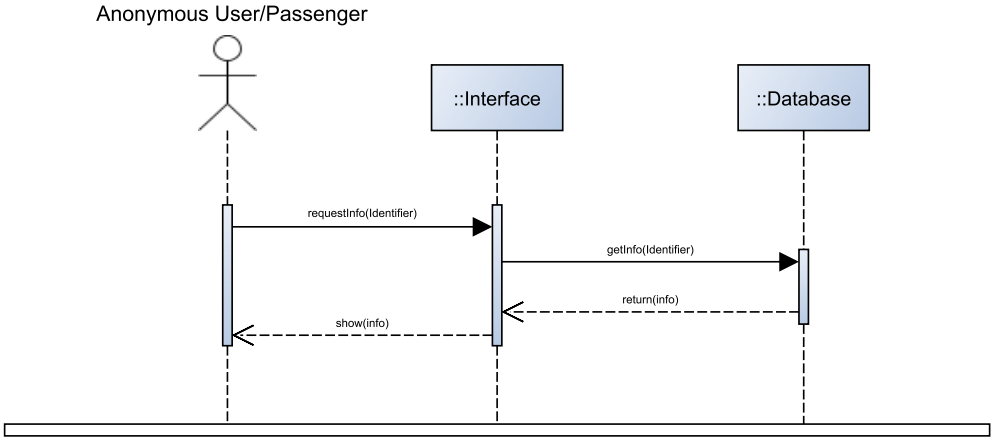
View Train and Station Details

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	View Train and Station Details		
Release (id)	UC13, v0.1, 28/04/2026		
Context of Use	Info View		
Scope	Let users view information of trains and stations. FR17, FR18, FR19 NFR10		
Level	Subfunction		
Primary actor	Anonymous User or Passenger		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Anonymous User/Passenger	View detailed information	
	System	Manages the user interface, the database and their interactions	
Preconditions	None		
Minimal Guarantees	None		
Success Guarantees	The user views the train service's or station's data		
Trigger	The user selects an abbreviated form of station or train service information		
Description	**Step**	**Action**	
	1	The user selects the abbreviated information	
	2A	The system displays the station's details	
	3A	The system displays an abbreviated form of train information for all trains passing through the station	
	4A	If the user is a Passenger	
	4A.1	the system also computes and provides options for tickets departing from the station	
	2B	The system provides the train's details	
	3B	The system displays an abbreviated form of station information for all stations the train passes through	
	4B	If the user is a Passenger	
	4B.1	The system provides options for tickets for the train	

	5	The user may select any abbreviated information to expand into another UC13 View Train and Station Details
	6	The user may select displayed tickets to expand into U8 View Ticket Details
Extension	**Step**	**Other action**
		None
Technology & data Variations	If the information belongs to a station, the path A is chosen. Otherwise, if the information belongs to a train service, path B is chosen.	





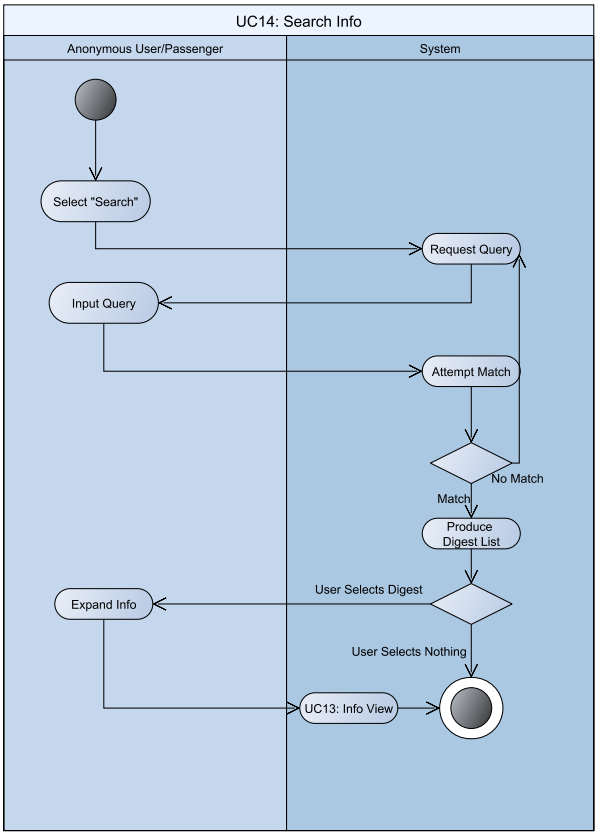
UC13: View Train and Station Details

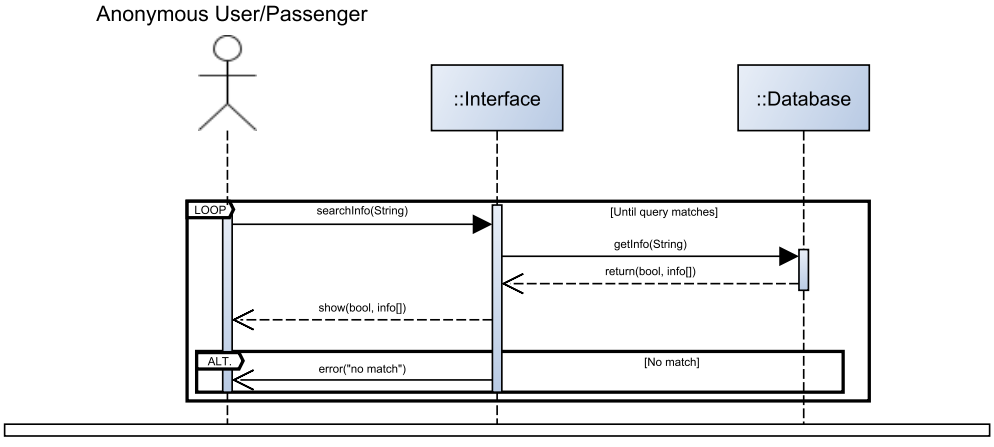
Search Info

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

<table>
<tr><td>Use case</td><td colspan="3">Search Info</td></tr>
<tr><td>Release (id)</td><td colspan="3">UC14, v0.1, 28/04/2026</td></tr>
<tr><td>Context of Use</td><td colspan="3">Info View</td></tr>
<tr><td>Scope</td><td colspan="3">Let users find station by name or Train service by ID.
FR17, FR18</td></tr>
<tr><td>Level</td><td colspan="3">Primary Task</td></tr>
<tr><td>Primary actor</td><td colspan="3">Anonymous user or Passenger</td></tr>
<tr><td rowspan="3">Stakeholder & interest</td><td colspan="2">Stakeholder</td><td>Interest</td></tr>
<tr><td colspan="2">Anonymous user/Passenger</td><td>Find information by query</td></tr>
<tr><td colspan="2">System</td><td>Manages the user interface, the database and their interactions</td></tr>
<tr><td>Preconditions</td><td colspan="3">None</td></tr>
<tr><td>Minimal Guarantees</td><td colspan="3">None</td></tr>
<tr><td>Success Guarantees</td><td colspan="3">The User is given a list of matching results, or is warned that there are no matches otherwise</td></tr>
<tr><td>Trigger</td><td colspan="3">User selects “Search”</td></tr>
<tr><td rowspan="8"></td><td>Step</td><td colspan="2">Action</td></tr>
<tr><td>1</td><td colspan="2">The user selects “Search”</td></tr>
<tr><td>2</td><td colspan="2">The system asks the user Station name or train ID for the query</td></tr>
<tr><td>3</td><td colspan="2">The user provides a string</td></tr>
<tr><td>4</td><td colspan="2">The system matches that string against all known names of stations and train service IDs</td></tr>
<tr><td>5</td><td colspan="2">For all matching stations/trains, a reduced set of their information is prepared and appended to a return list</td></tr>
<tr><td>6</td><td colspan="2">The system provides the user with the return list</td></tr>
<tr><td>7</td><td colspan="2">The user may select the abbreviated information to expand to a full UC13 View Train and Station Details</td></tr>
<tr><td rowspan="2">Extension</td><td>Step</td><td colspan="2">Other action</td></tr>
<tr><td>4a</td><td colspan="2">If no name matches the string</td></tr>
</table>

	4a.1	Produce “No match” error and return to step 2
Technology & data Variations	None	



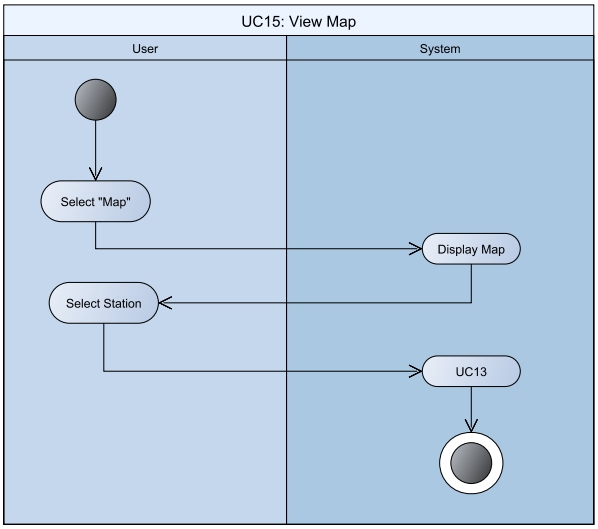


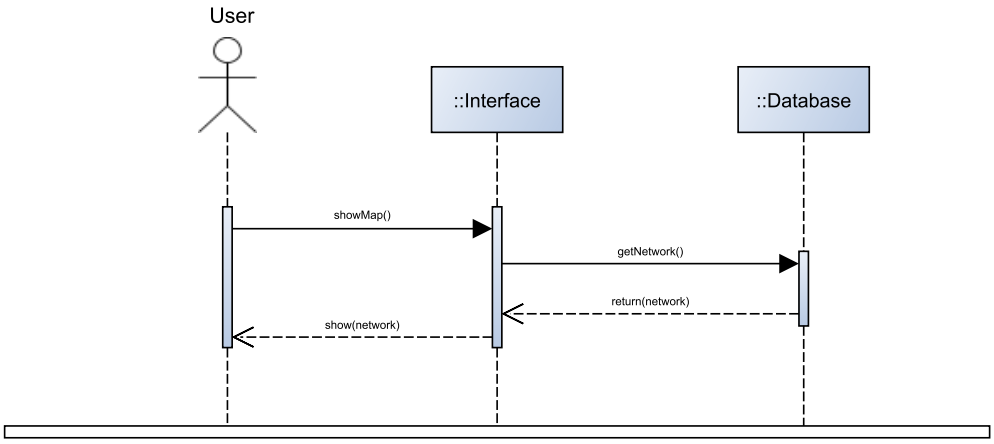
UC14: Search Info

View Map

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

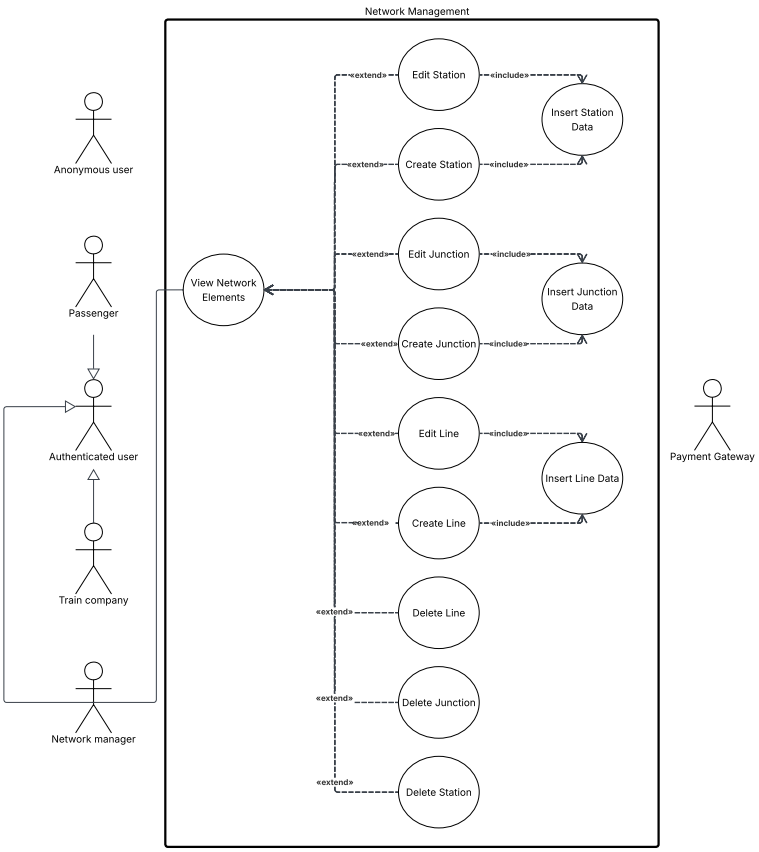
Use case	View Map	
Release (id)	UC15, v0.1, 28/04/2026	
Context of Use	Info View	
Scope	Let any user see the stations and their connections. FR23	
Level	Primary Task	
Primary actor	Anonymous or Authenticated user	
Stakeholder & interest	**Stakeholder**	**Interest**
	Anonymous/Authenticated user	See topology
	System	Manages the user interface, the database and their interactions
Preconditions	None	
Minimal Guarantees	None	
Success Guarantees	The User view stations distributed according to the topology of the network, and select one to expand into UC13 View Train and Station Details	
Trigger	User selects “Show Map”	
Description	**Step**	**Action**
	1	The user selects “Show Map”
	2	The system displays the topological map
	3	If the user selects a station, initiate UC13: View Train and Station Details for that station
Extension	**Step**	**Other action**
		None
Technology & data Variations	None	





UC15: View Map

Network management

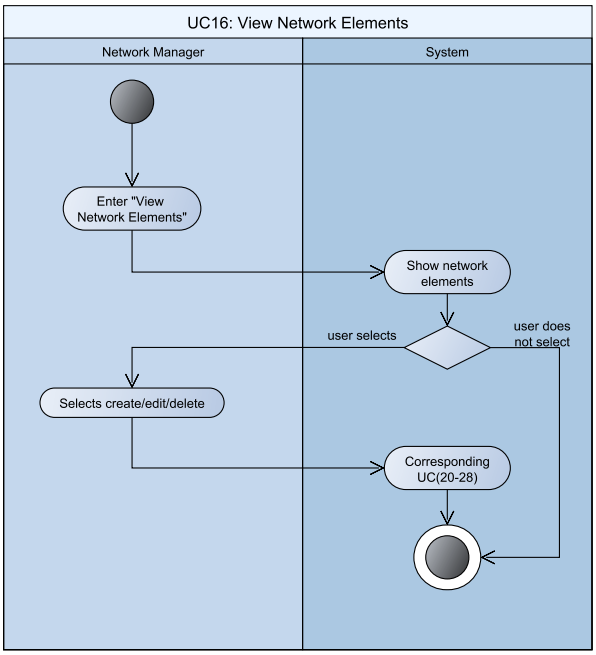


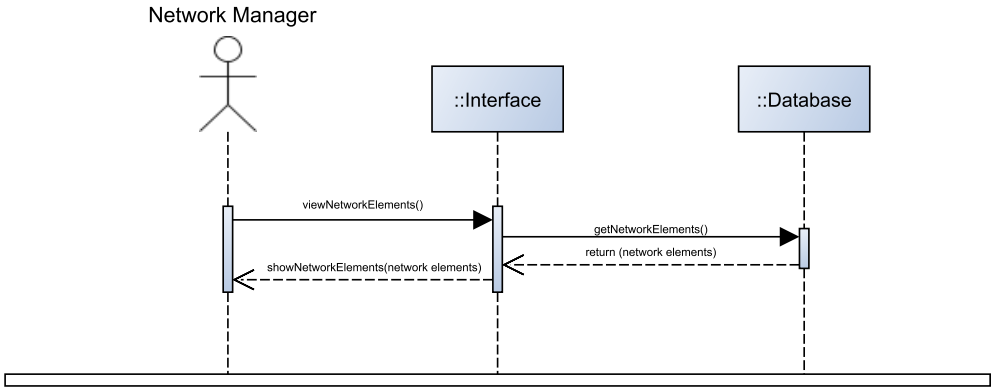
Original image can be found on our Github here

View Network Elements

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**View Network Elements**	
Release (id)	UC16, v0.3, 03/05/2026	
Context of Use	Network Management	
Scope	Let the Network Manager view the elements of the network. FR24, FR25, FR26 NFR14	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to view the elements that make up the network
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated.	
Minimal Guarantees	None	
Success Guarantees	The network elements are displayed	
Trigger	The Network Manager enters Network Management mode.	
Description	**Step**	**Action**
	1	Display a list of all network elements (Stations, Junctions, Lines), with their information.
Extensions	**Step**	**Other Action**
	1a	If the Network Manager selects edit on any network element, depending on the type of network element, go to one of: UC23:Edit Station, UC24: Edit Junction, UC25: Edit Line
	1b	If the Network Manager selects delete on any network element, depending on the type of network element, go to one of: UC26: Delete Station, UC27: Delete Junction, UC28: Delete Line
	1c	If the Network Manager selects any create option, go to the respective use case: UC20: Create Station, UC21: Create Junction, UC22: Create Line
Technology & data Variations	None	



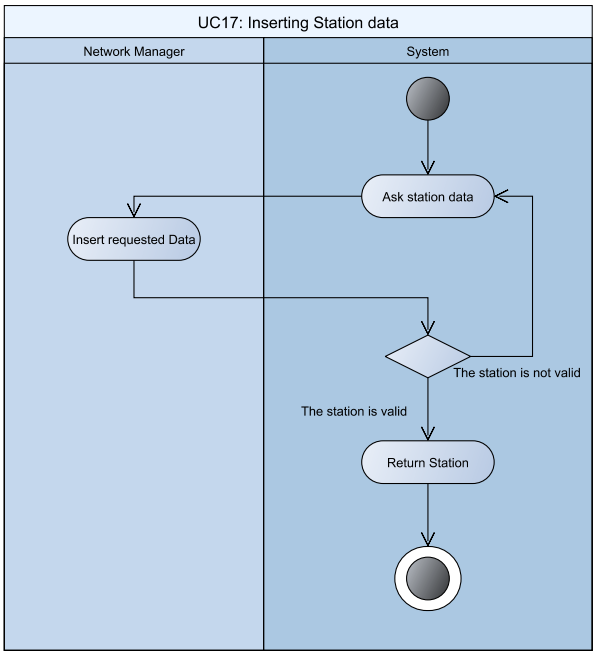


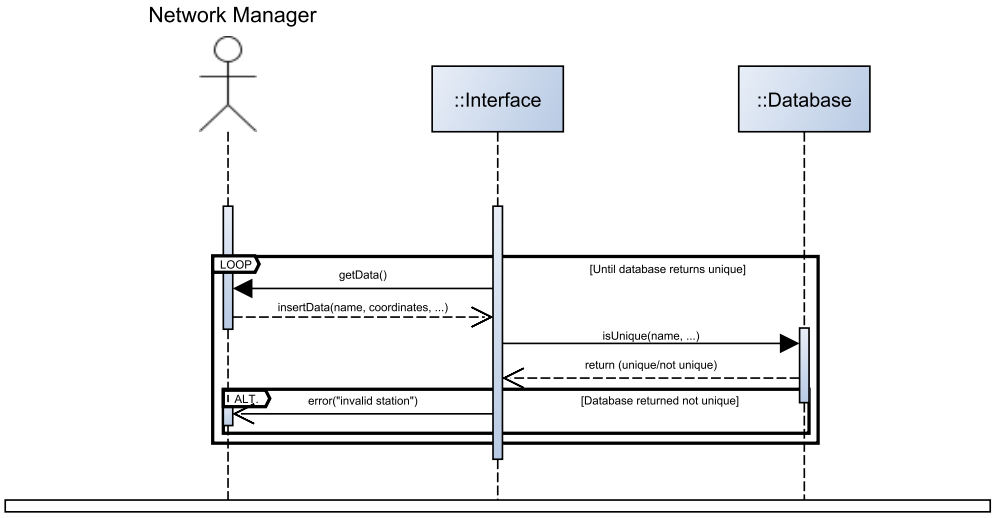
UC16: View Network Elements

Insert Station Data

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**Insert Station Data**	
Release (id)	UC17, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the System provide a standardised way to insert a Station's data. FR24 NFR14	
Level	Subfunction	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to insert Station's Data
	System	Manages the user interface, the database and their interactions
Preconditions	None	
Minimal Guarantees	No invalid Station is returned to the calling subsystem	
Success Guarantees	A valid Station is returned to the calling subsystem	
Trigger	Another subsystem includes this use case	
Description	**Step**	**Action**
	1	The system asks the user to insert the Station's information, that are: name, coordinates, platforms (including their locally unique identifier, e.g. "1" or "1-East")
	2	The user inserts the requested data.
	3	The created station is returned to the calling subsystem
Extensions	**Step**	**Alternate Action**
	2a	If the Station's name is not unique (also amongst Junctions) or the platforms identifiers are not locally unique
	2a.1	Display an error
	2a.2	Return to step 1



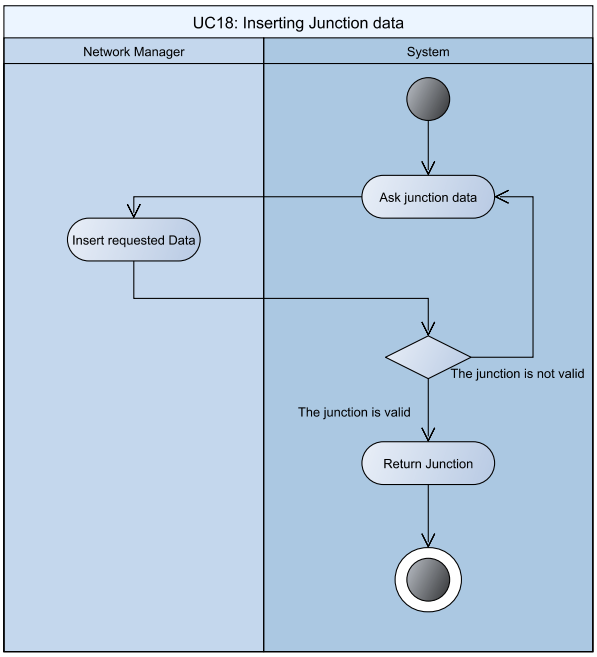


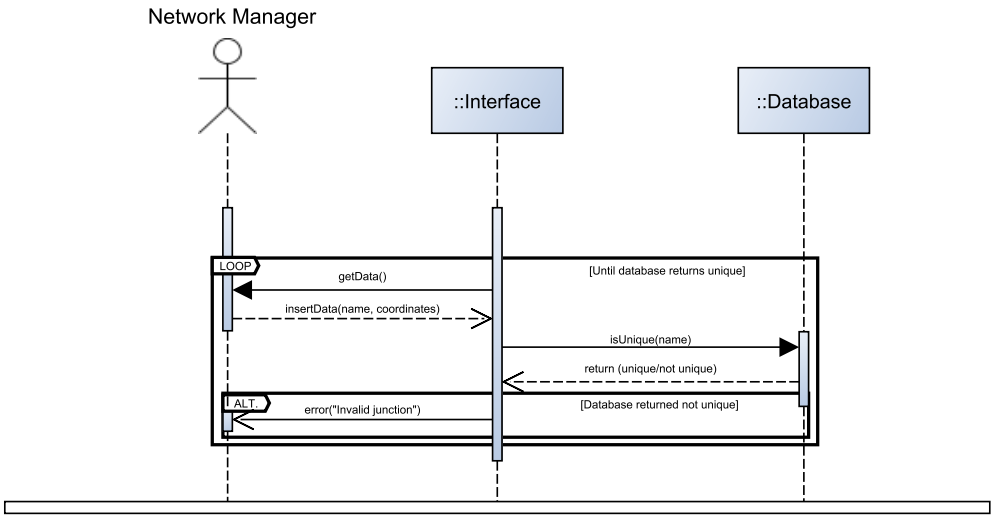
UC17: Inserting Station Data

Insert Junction Data

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use Case	Insert Junction Data	
Release (id)	UC18, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the System provide a standardised way to insert a Junction's data. FR26 NFR14	
Level	Subfunction	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to insert Junction's Data
	System	Manages the user interface, the database and their interactions
Preconditions	None	
Minimal Guarantees	No invalid Junction is returned to the calling subsystem	
Success Guarantees	A valid Junction is returned to the calling subsystem	
Trigger	Another subsystem includes this use case	
Description	**Step**	**Action**
	1	The system asks the user to insert the Junction information, that are: name and coordinates.
	2	The user inserts the requested data.
	3	The Junction is returned to the calling subsystem
Extensions	**Step**	**Alternate action**
	2a	If the Junction's name is not unique (also amongst Stations)
	2a.1	Display an error
	2a.2	Go to step 1
Technology & data Variations	None	



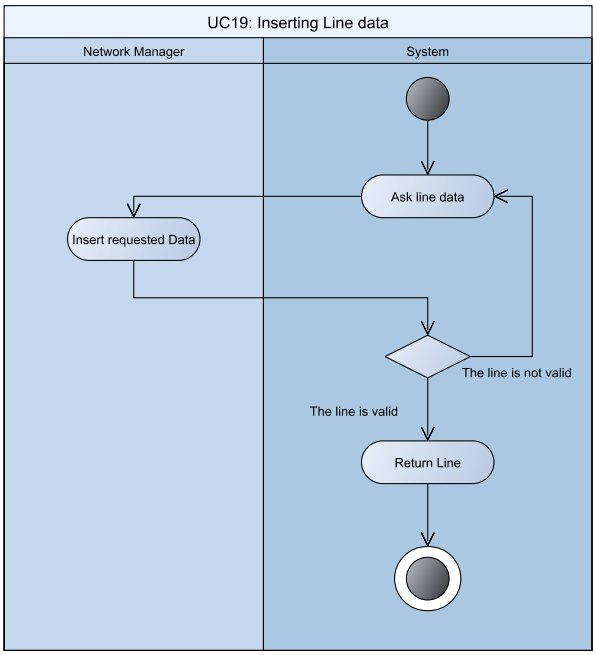


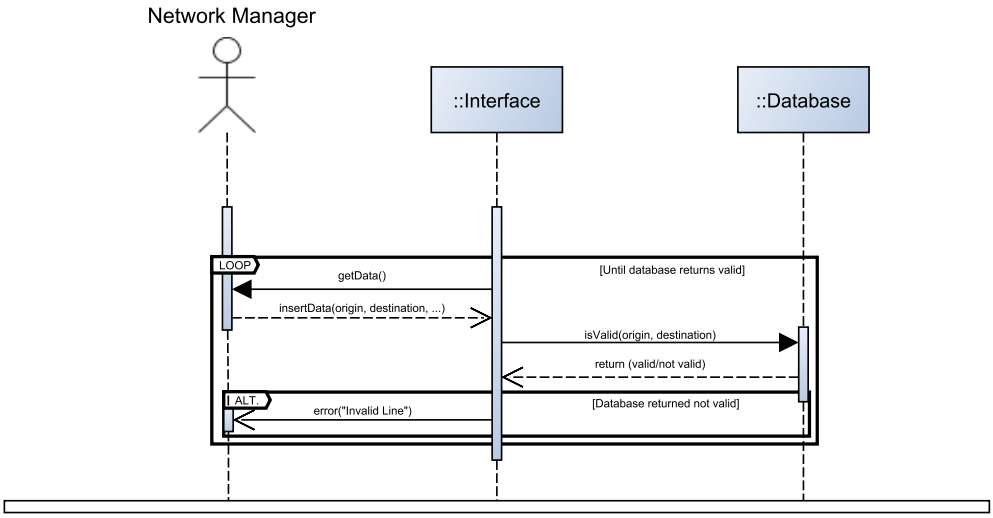
UC18: Inserting Junction Data

Insert Line Data

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**Insert Line Data**	
Release (id)	UC19, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the System provide a standardised way to insert a Line's data. FR25 NFR14	
Level	Subfunction	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to insert Line's Data
	System	Manages the user interface, the database and their interactions
Preconditions	None	
Minimal Guarantees	No invalid Line is returned to the calling subsystem	
Success Guarantees	A valid Line is returned to the calling subsystem	
Trigger	Another subsystem includes this use case	
Description	**Step**	**Action**
	1	The system asks the user to insert the Line's information, that are: origin, destination, number of tracks, length, maximum speed
	2	The user inserts the requested data.
	3	The system returns the Line to the calling subsystem
Extensions	**Step**	**Other Action**
	2a	If the origin or the destination do not exist
	2a.1	Display an error
	2a.2	Go to step 1
Technology & data Variations	None	



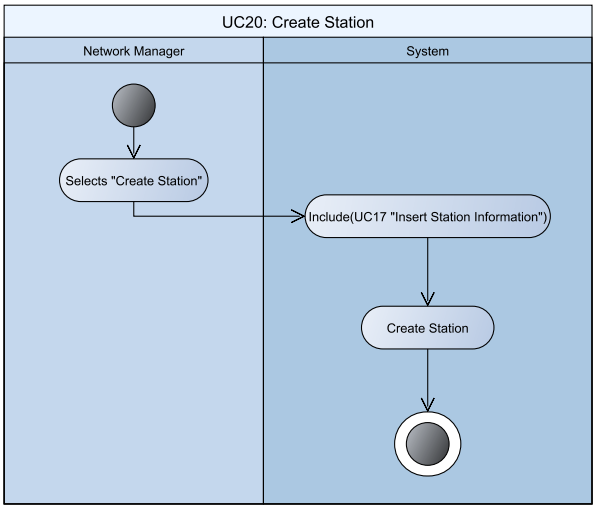


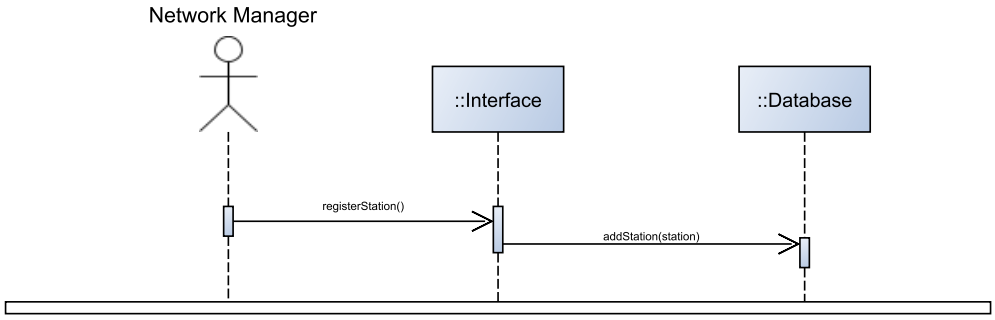
UC19: Inserting Line Data

Create Station

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**Create Station**		
Release (id)	UC20, v0.3, 26/04/2026		
Context of Use	Network Management		
Scope	Let the network manager create a Station. FR24 NFR14		
Level	Primary task		
Primary actor	Network Manager		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Network Manager	Wants to create a Station	
	System	Manages the user interface, the database and their interactions	
Preconditions	The Network Manager is authenticated		
Minimal Guarantees	No invalid Station is registered		
Success Guarantees	A Station is registered		
Trigger	The Network Manager selects “Register a station”.		
Description	**Step**	**Action**	
	1	The Network Manager selects “Register a station”.	
	2	Include(UC17 Insert Station Data)	
	3	The system registers the Station.	
Extensions	**Step**	**Other Action**	
	-	None	
Technology & data Variations	None		



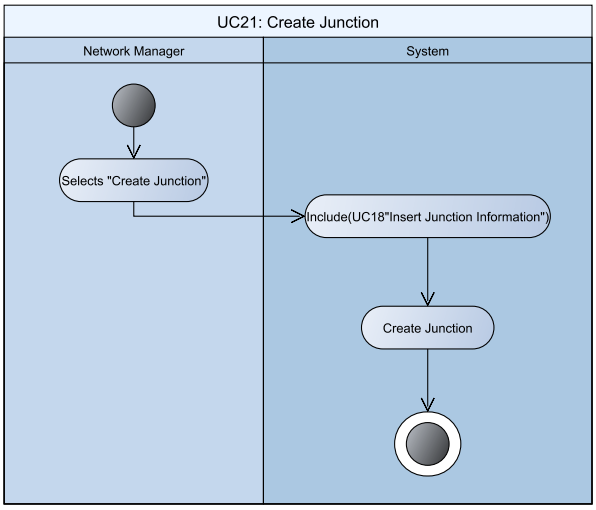


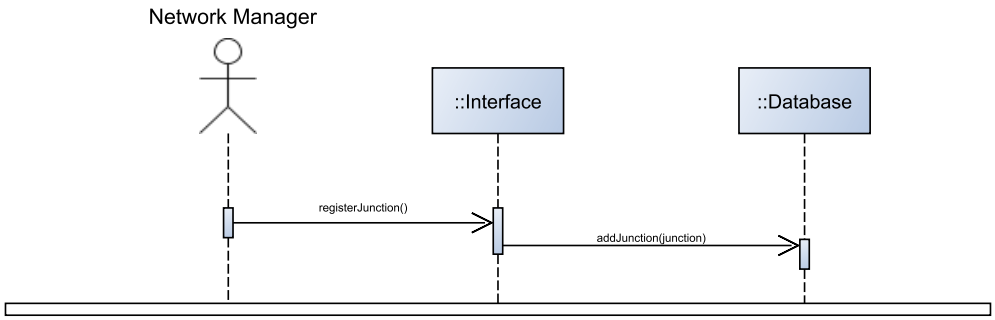
UC20: Create Station

Create Junction

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use Case	Create Junction		
Release (id)	UC21, v0.3, 26/04/2026		
Context of Use	Network Management		
Scope	Let the network manager create a Junction. FR26 NFR14		
Level	Primary task		
Primary actor	Network Manager		
Stakeholder & interest	**Stakeholder**		**Interest**
	Network Manager		Wants to create a Junction
	System		Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated		
Minimal Guarantees	No invalid Junction is registered		
Success Guarantees	A Junction is registered		
Trigger	The Network Manager selects ”Register a junction”.		
Description	**Step**	**Action**	
	1	The Network Manager selects ”Register a junction”.	
	2	Include(UC18 Insert Junction Data)	
	3	The system registers the Junction.	
Extensions	**Step**	**Other Action**	
	-	None	
Technology & data Variations	None		



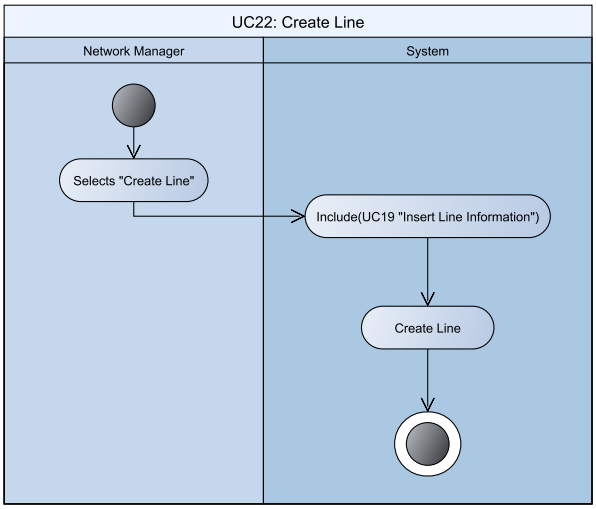


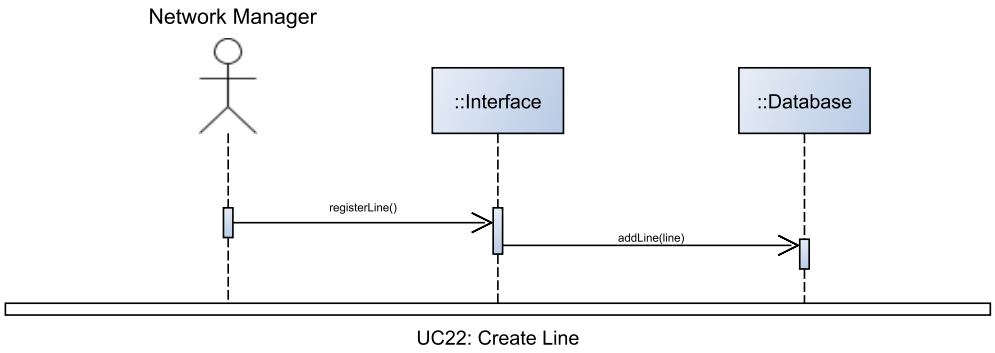
UC21: Create Junction

Create Line

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Create Line	
Release (id)	UC22, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the network manager create Lines. FR25 NFR14	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to create a Line
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated	
Minimal Guarantees	No invalid Line is registered	
Success Guarantees	A Line is registered	
Trigger	The Network Manager selects “Register a line”.	
Description	**Step**	**Action**
	1	The Network Manager selects “Register a line”.
	2	Include(UC19 Insert Line Data)
	3	The system registers the Line.
Extensions	**Step**	**Other Action**
	-	None
Technology & data Variations	None	



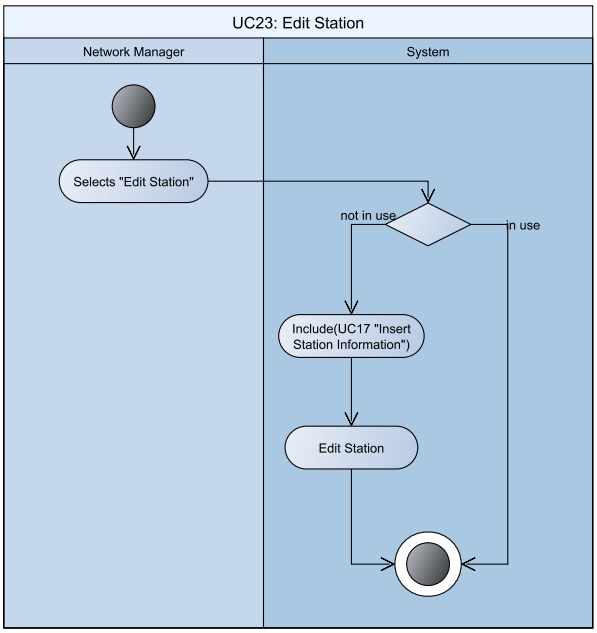


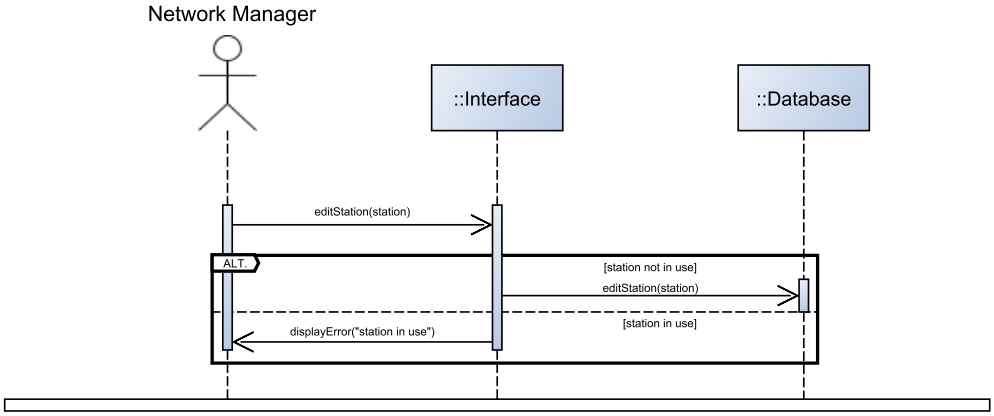
UC22: Create Line

Edit Station

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use Case	Edit Station	
Release (id)	UC23, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the network manager edit a Station. FR24 NFR14	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to edit a Station
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated	
Minimal Guarantees	No service information is invalidated	
Success Guarantees	The selected Station is edited	
Trigger	The Network Manager selects “Edit station”.	
Description	**Step**	**Action**
	1	The Network Manager selects “Edit station”.
	2	If the station is not used by any service
	2a.1	Include(UC17 Insert Station Data).
	2a.2	The system edits the Station with the data provided by step 2a.1.
	3	Else
	3a.1	Display an error to the user.
Extensions	**Step**	**Other Action**
	-	None
Technology & data Variations	None	



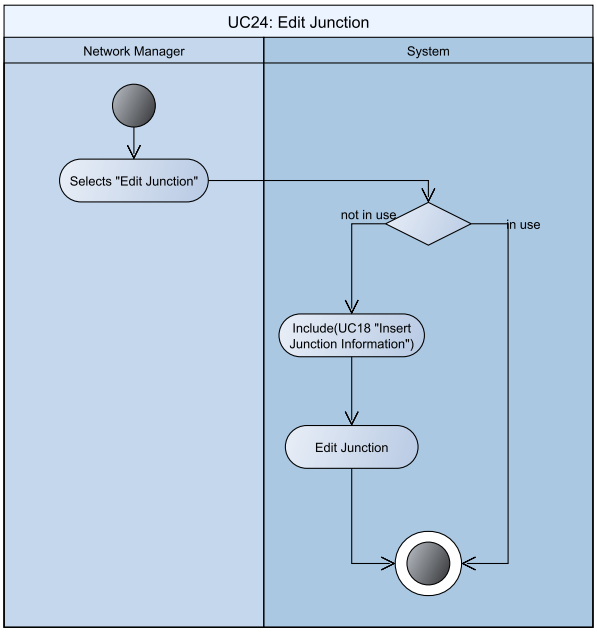


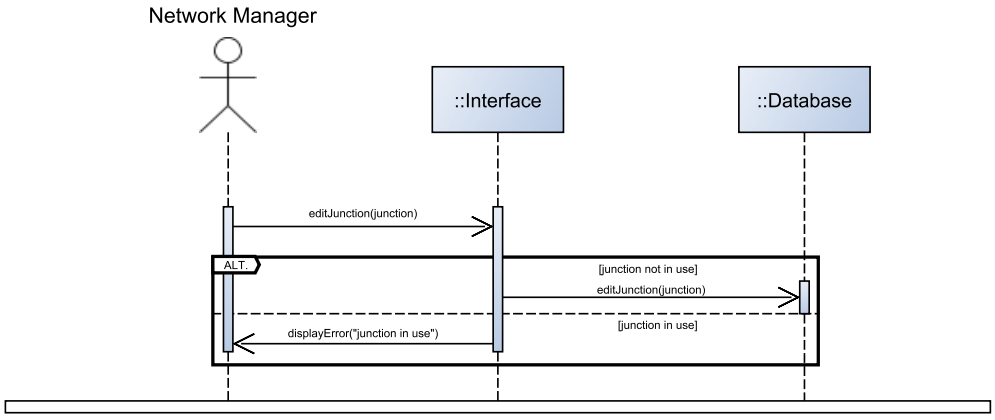
UC23: Edit Station

Edit Junction

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use Case	Edit Junction	
Release (id)	UC24, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the network manager edit a Junction. FR26 NFR14	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to edit a Junction
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated	
Minimal Guarantees	None	
Success Guarantees	The selected Junction is edited	
Trigger	The Network Manager selects “Edit junction”.	
Description	**Step**	**Action**
	1	The Network Manager selects “Edit junction”.
	2	If the junction is not used by any service
	2a.1	Include(UC18 Insert Junction Data)
	2a.2	The system edits the Junction with the data provided by step 2a.1
	3	Else
	3a.1	Display an error to the user
Extensions	**Step**	**Other Action**
	-	None
Technology & data Variations	None	



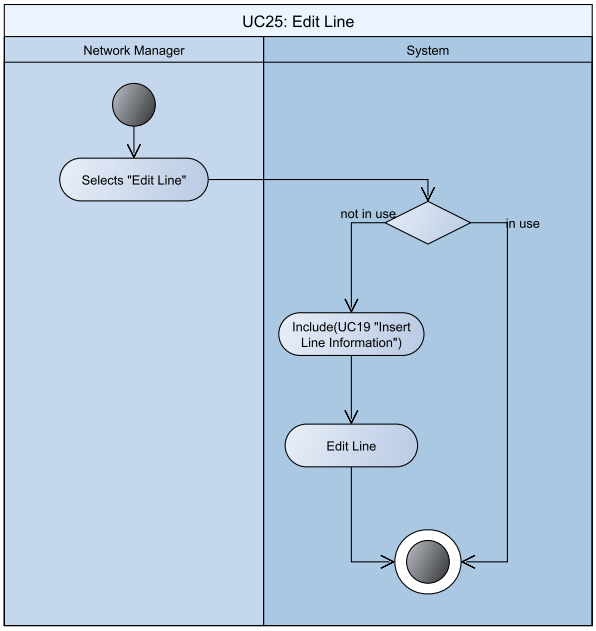


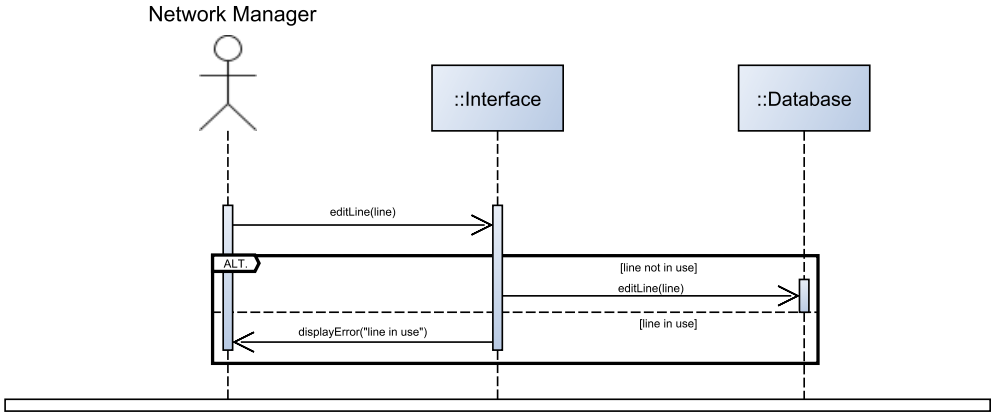
UC24: Edit Junction

Edit Line

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use Case	Edit Line	
Release (id)	UC25, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the network manager edit a Line. FR25 NFR14	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to edit a Line
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated	
Minimal Guarantees	No service information is invalidated	
Success Guarantees	The selected Line is edited	
Trigger	The Network Manager selects “Edit line”.	
Description	**Step**	**Action**
	1	The Network Manager selects “Edit line”.
	2	If the line is not used by any service
	2a.1	Include(UC19 Insert Line Data)
	2a.2	The system edits the Line with the data provided by step 2a.1
	3	Else
	3a	Display an error to the user
Extensions	**Step**	**Other Action**
	-	None
Technology & data Variations	None	





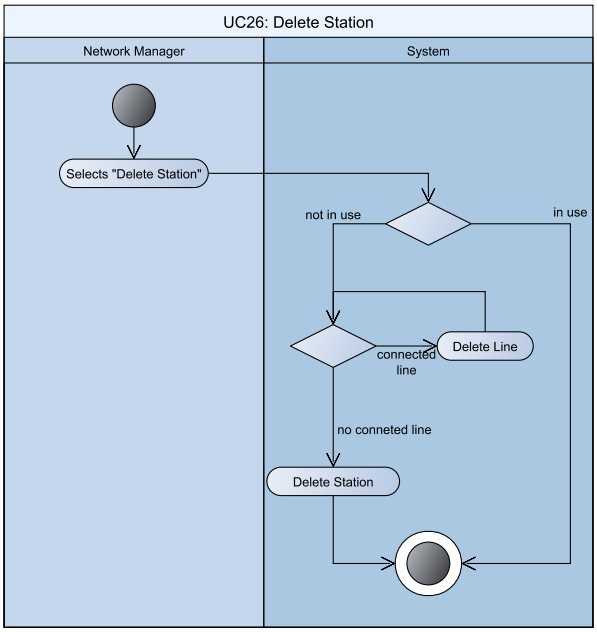
UC25: Edit Line

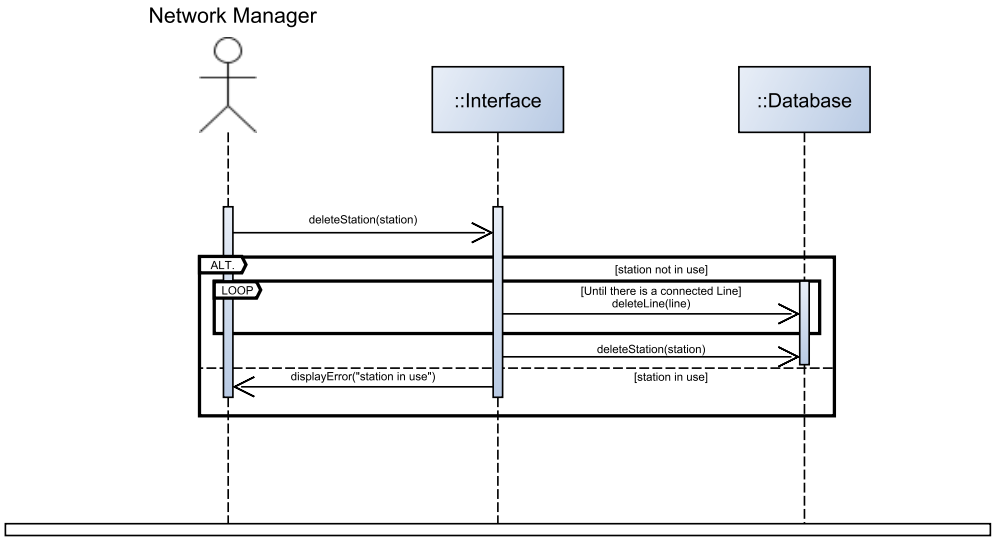
Delete Station

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	Delete Station	
Release (id)	UC26, v0.3, 26/04/2026	
Context of Use	Network Management	
Scope	Let the network manager delete a station. FR24 NFR14	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Network Manager	Wants to delete a station
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated	
Minimal Guarantees	No service information is invalidated	
Success Guarantees	A station and it's connected lines (if any) are deleted	
Trigger	The Network Manager select "Delete station".	

Description	**Step**	**Action**
	1	The Network Manager select "Delete station".
	2	The system checks if any Service is using this station
	3	Foreach Line connected to this station
	3a	The line is deleted
	4	The station is deleted
Extensions	**Step**	**Other Action**
	2a	If any service is scheduled to use this station
	2a.1	Display an error informing the user that the station cannot be deleted as it is in use
	2a.2	Conclude use case early
Technology & data Variations	None	



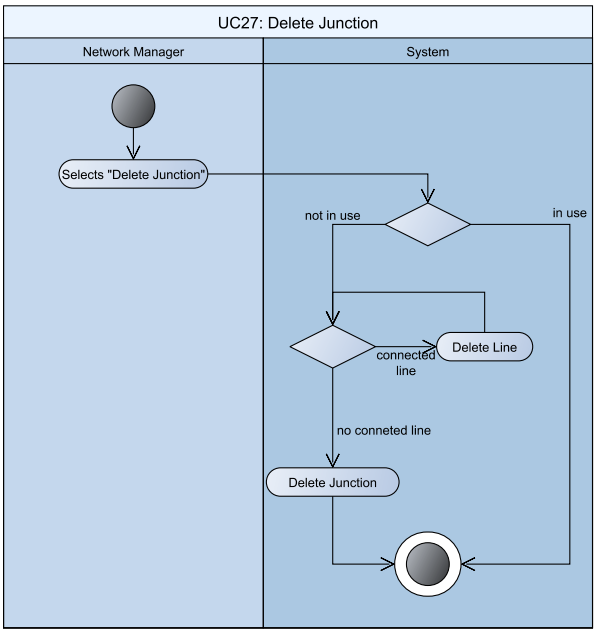


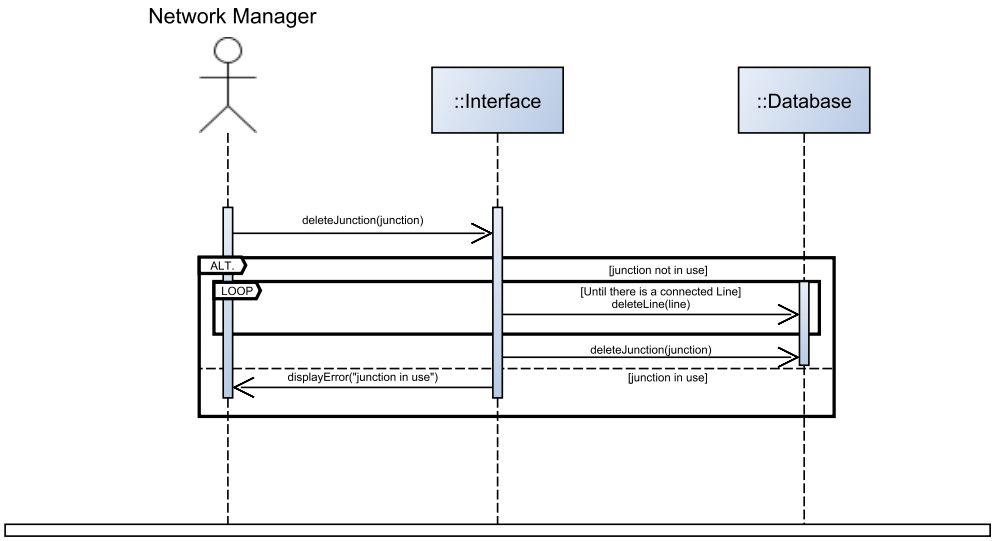
UC26: Delete Station

Delete Junction

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use Case	Delete Junction		
Release (id)	UC27, v0.3, 26/04/2026		
Context of Use	Network Management		
Scope	Let the network manager Delete a junction. FR26 NFR14		
Level	Primary task		
Primary actor	Network Manager		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Network Manager	Wants to delete a junction	
	System	Manages the user interface, the database and their interactions	
Preconditions	The Network Manager is authenticated		
Minimal Guarantees	No service information is invalidated		
Success Guarantees	A junction and it's connected lines (if any) are deleted		
Trigger	The Network Manager selects "delete junction".		
Description	**Step**	**Action**	
	1	The Network Manager selects "delete junction".	
	2	The system checks if any Service is using this junction	
	3	Foreach line connected to this junction	
	3a	The line is deleted	
	4	The junction is deleted	
Extensions	**Step**	**Other Action**	
	2a	If any service is scheduled to use the junction	
	2a.1	Display an error	
	2a.2	Conclude use case early	
Technology & data Variations	None		



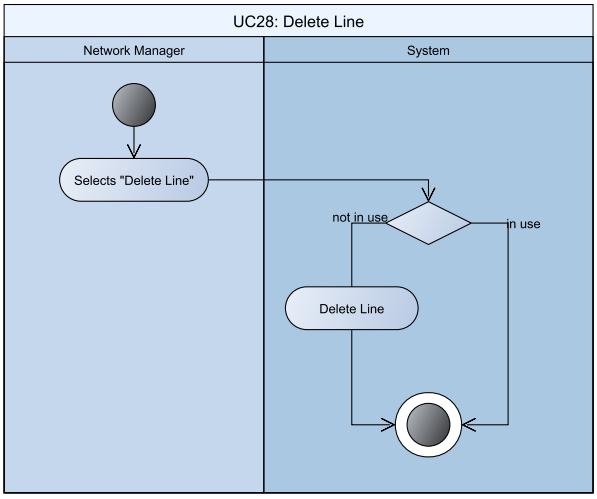


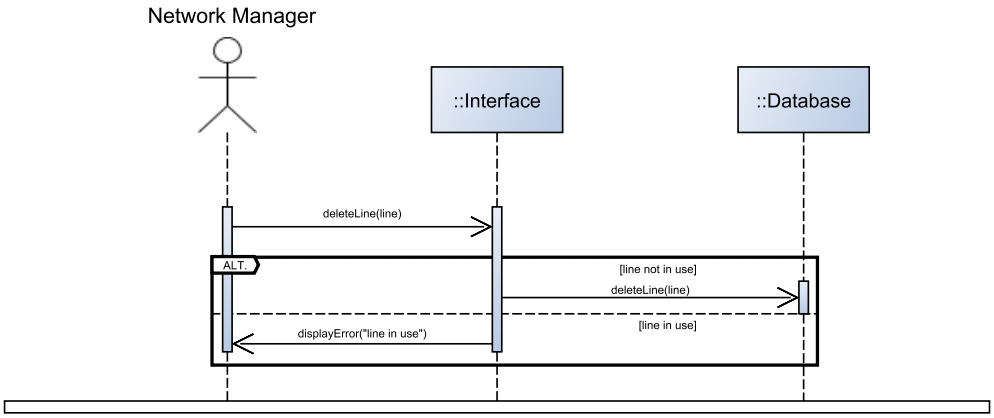
UC27: Delete Junction

Delete Line

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

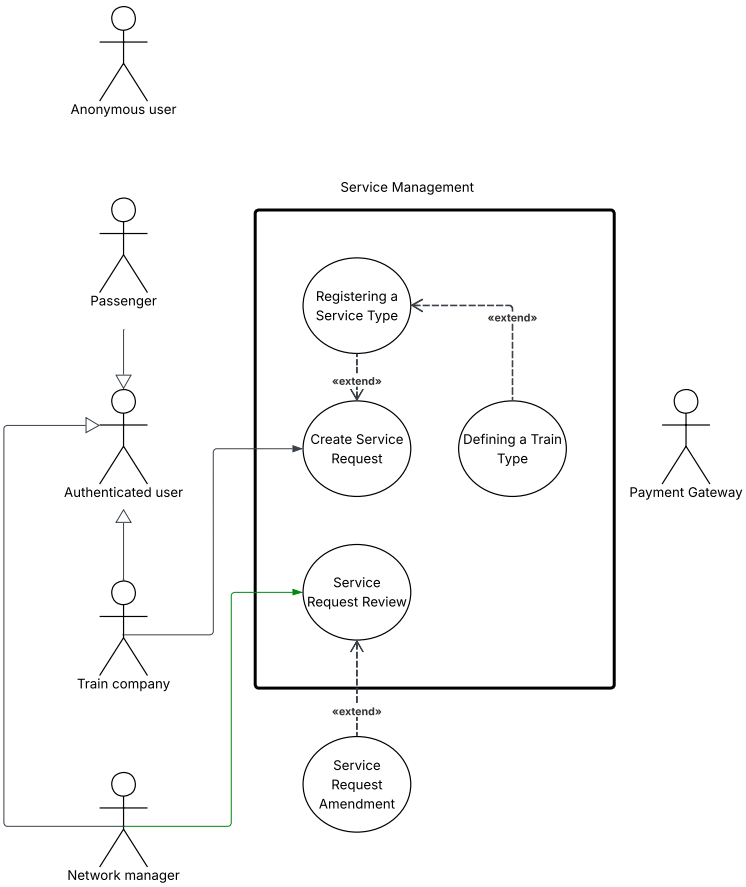
Use case	**Delete Line**		
Release (id)	UC28, v0.3, 26/04/2026		
Context of Use	Network Management		
Scope	Let the network manager delete a line. FR25 NFR14		
Level	Primary task		
Primary actor	Network Manager		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Network Manager	Wants to delete a Line	
	System	Manages the user interface, the database and their interactions	
Preconditions	The Network Manager is authenticated		
Minimal Guarantees	No service information is invalidated		
Success Guarantees	A valid Line is deleted		
Trigger	The Network Manager selects “delete line”.		
Description	**Step**	**Action**	
	1	The Network Manager selects “delete line”.	
	2	The system checks if any Service is scheduled to use the Line	
	3	The line is deleted	
Extensions	**Step**	**Other Action**	
	2a	If any service is scheduled to use the line	
	2a.1	Display an Error	
	2a.2	Conclude use case early	
Technology & data Variations	None		





UC28: Delete Line

Service management

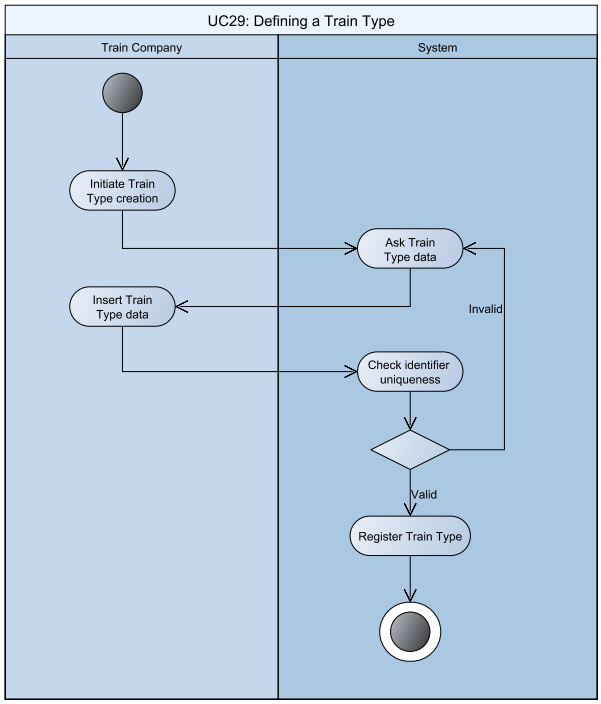


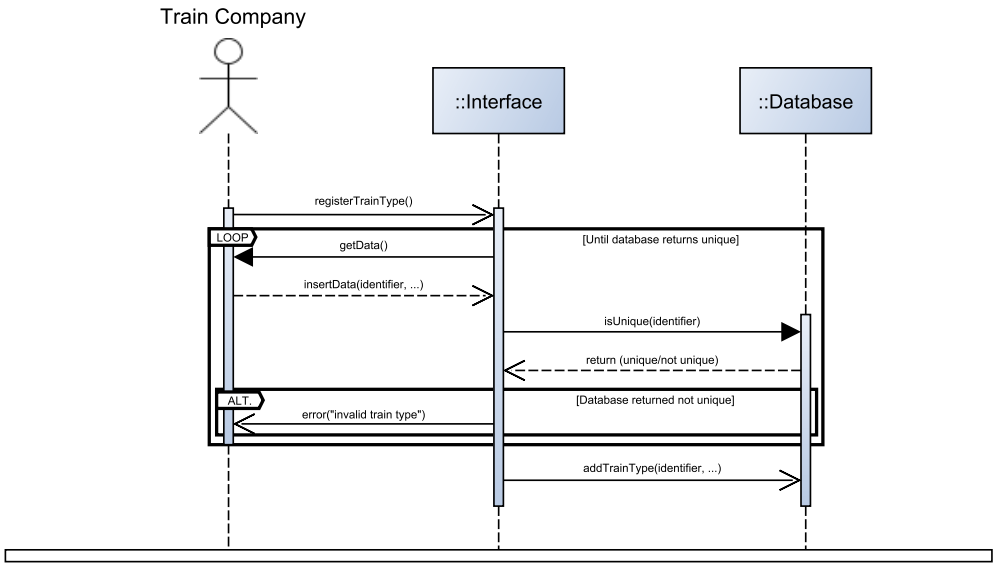
Original image can be found on our Github here

Defining a Train Type

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**Defining a Train Type**	
Release (id)	UC29, v0.3, 30/04/2026	
Context of Use	Service Management	
Scope	Let a Train Company register one of their trains for use in services. FR21 NFR14	
Level	Primary task	
Primary actor	Train Company	
Stakeholder & interest	**Stakeholder**	**Interest**
	Train Company	Wants to register one of their trains
	System	Manages the user interface, the database and their interactions
Preconditions	The Train Company is authenticated	
Minimal Guarantees	No train with duplicated identifier is registered	
Success Guarantees	A train type is registered	
Trigger	The Train Company initiates the creation of a Train Type.	
Description	**Step**	**Action**
	1	The Train Company initiates the creation of a Train Type.
	2	The system asks the user to insert the train information, that are: identifier, max speed, type (Cargo/Passenger) and, if Passenger, seated and standing Capacities
	3	The train is registered.
Extensions	**Step**	**Other Action**
	2a	If the Train Type identifier is not unique
	2a.1	Then display an error informing the user.
	2a.2	Return to step 2
Technology & data Variations	None	



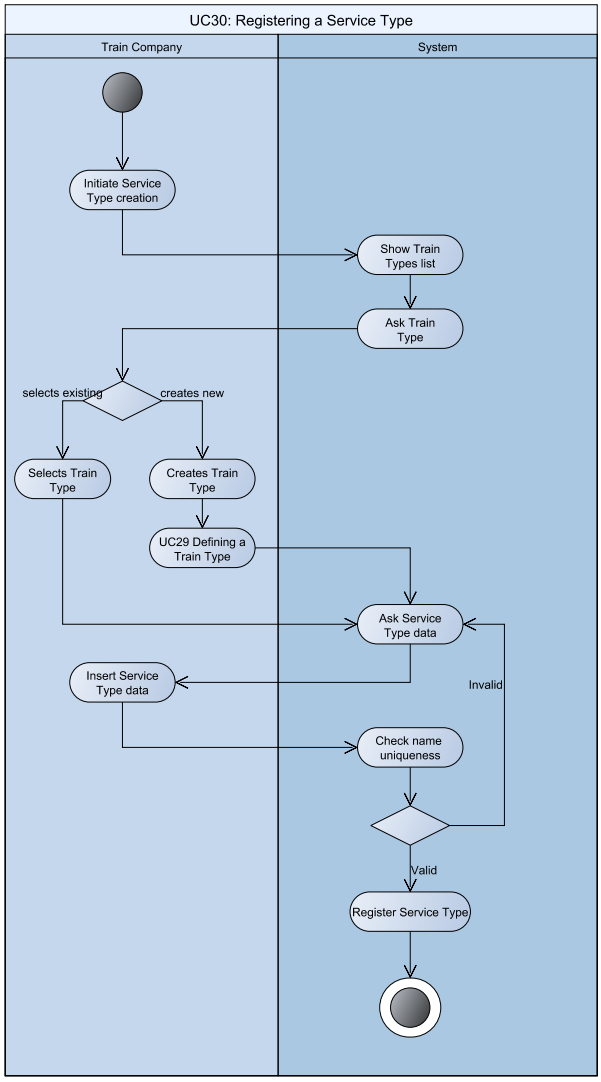


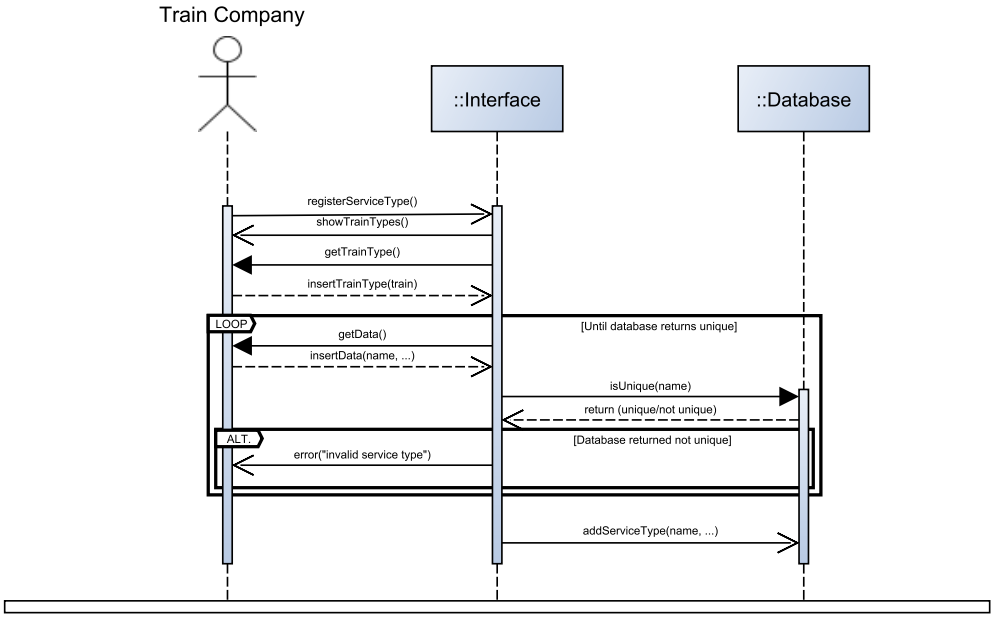
UC29: Defining a Train Type

Registering a Service Type

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**Registering a Service Type**	
Release (id)	UC30, v0.4, 03/05/2026	
Context of Use	Service Management	
Scope	Let a Train Company register a service type. FR22	
Level	Primary task	
Primary actor	Train Company	
Stakeholder & interest	**Stakeholder**	**Interest**
	Train Company	Wants to register a service Type
	System	Manages the user interface, the database and their interactions
Preconditions	The Train Company is authenticated	
Minimal Guarantees	No duplicated commercial name is registered	
Success Guarantees	A service is registered	
Trigger	The Train Company initiates the creation of a Service Type.	
Description	**Step**	**Action**
	1	The Train Company initiates the creation of a Service Type
	2	The system shows a list of registered Train Types.
	3	The system asks the user to select an existing or create a new Train Type.
	4	The user selects the Train Type.
	5	The system asks the user to insert the service information, that are: commercial name, price / km
	6	The user inserts the required data
	7	The service type is registered.
Extensions	**Step**	**Other Action**
	4a	If the user decides to create a new Train Type
	4a.1	Then UC29 (Defining a Train Type).
	4a.2	Select the newly created Train Type.
	5a	If the service's commercial name is not unique
	5a.1	Then display an error informing the user.
	5a.2	Return to step 5


UC30: Registering a Service Type
Train Company
System
Initiate Service Type creation
Show Train Types list
Ask Train Type
selects existing
creates new
Selects Train Type
Creates Train Type
UC29 Defining a Train Type
Ask Service Type data
Insert Service Type data
Invalid
Check name uniqueness
Valid
Register Service Type



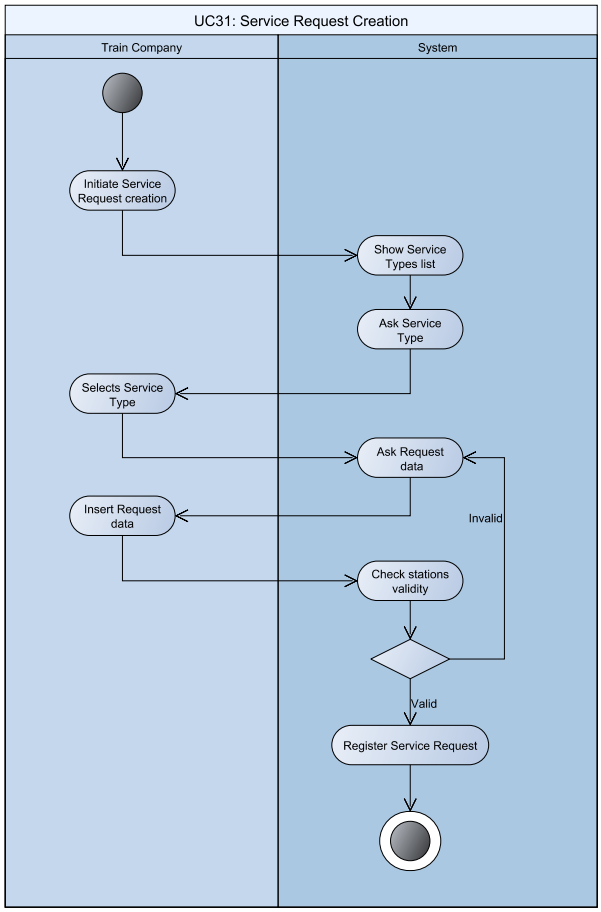
UC30: Registering a Service Type

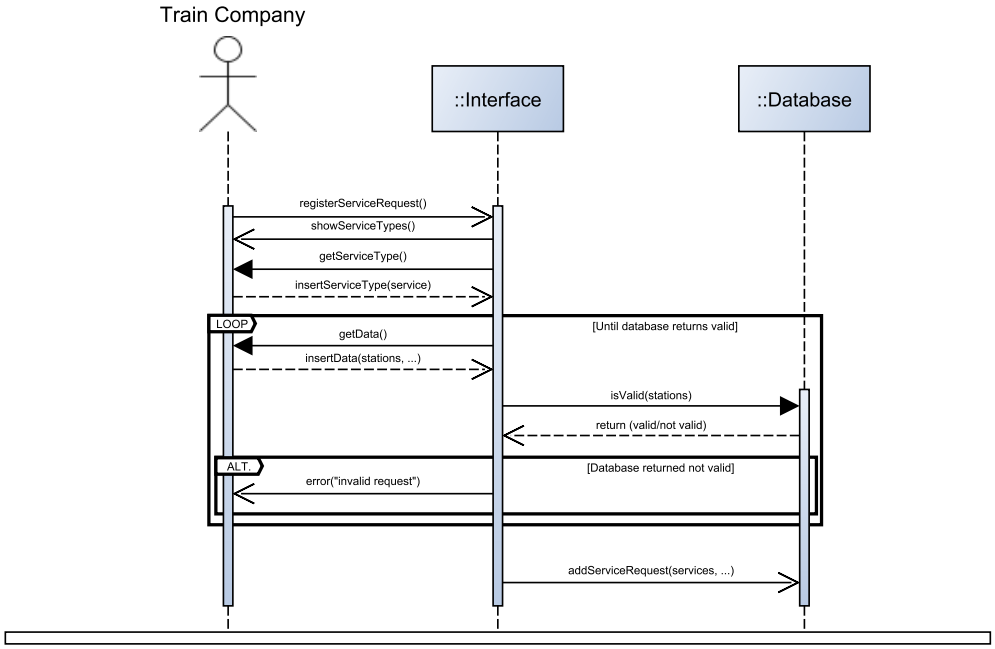
Create Service Request

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

Use case	**Create Service Request**		
Release (id)	UC31, v0.1, 03/05/2026		
Context of Use	Service Management		
Scope	Let a Train Company create a service request. FR20		
Level	Primary task		
Primary actor	Train Company		
Stakeholder & interest	**Stakeholder**	**Interest**	
	Train Company	Wants to create a service request	
	System	Manages the user interface, the database and their interactions	
Preconditions	The Train Company is authenticated		
Minimal Guarantees	No invalid service is created		
Success Guarantees	A service is created		
Trigger	The Train Company initiates the creation of a Service Request.		
Description	**Step**	**Action**	
	1	The Train Company initiates the creation of a Service Request.	
	2	The system shows a list of registered Service Types.	
	3	The system asks the user to select an existing or create a new Service Type.	
	4	The user selects the Service Type.	
	5	The system asks the user to insert the request information, that are: a list of steps and the time taken between them; a list of dispatch times (made of time, Train number, days of the week)	
	6	The service request is created.	
Extensions	**Step**	**Other Action**	
	4a	If the user decides to create a new Service Type	
	4a.1	Then UC30: Defining a Train Type	
	4a.2	Select the newly created Train Type.	
	5a.1	If the user did not insert at least 2 steps or the train numbers are not unique in the request	

	5a.1.1	Display an error
	5a.1.2	Go to step 5
Technology & data Variations	None	


UC31: Service Request Creation
Train Company
System
Initiate Service Request creation
Show Service Types list
Ask Service Type
Selects Service Type
Ask Request data
Insert Request data
Invalid
Check stations validity
Valid
Register Service Request

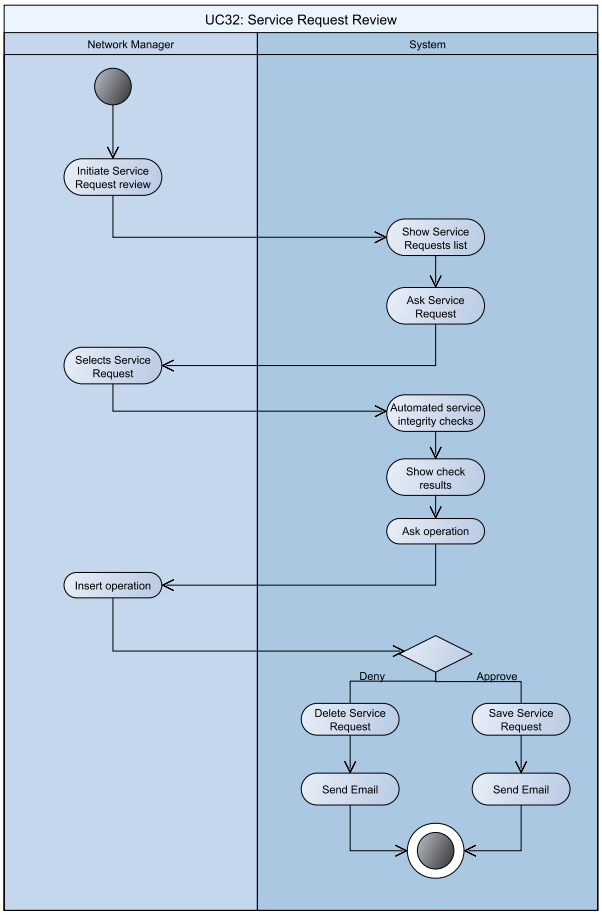


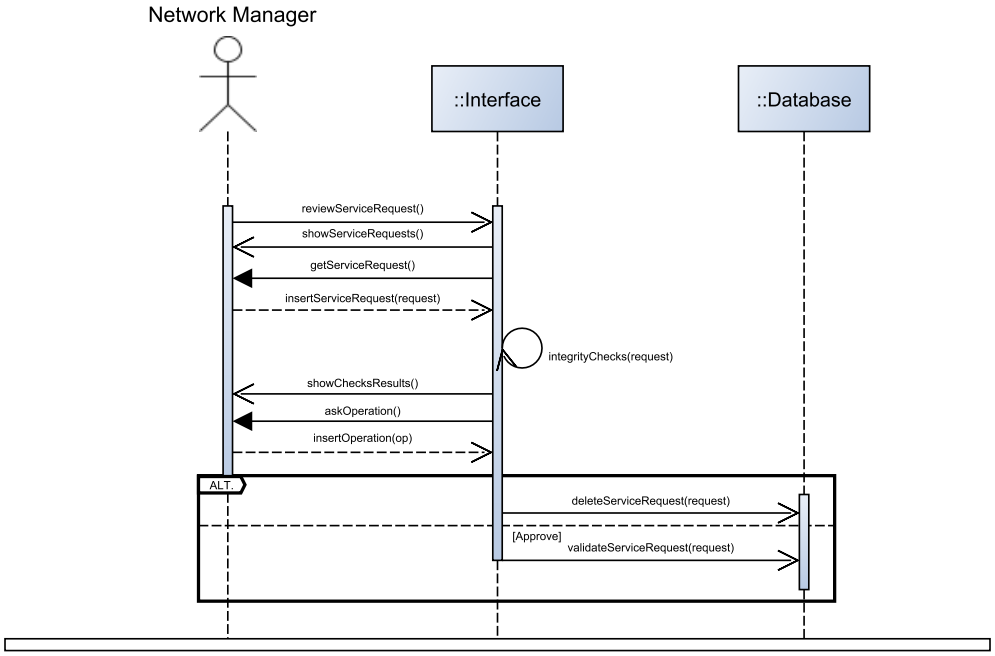
UC31: Service Request Creation

Service Request Review

Original Activity diagram image can be found on our Github here
Original Sequence diagram image can be found on our Github here

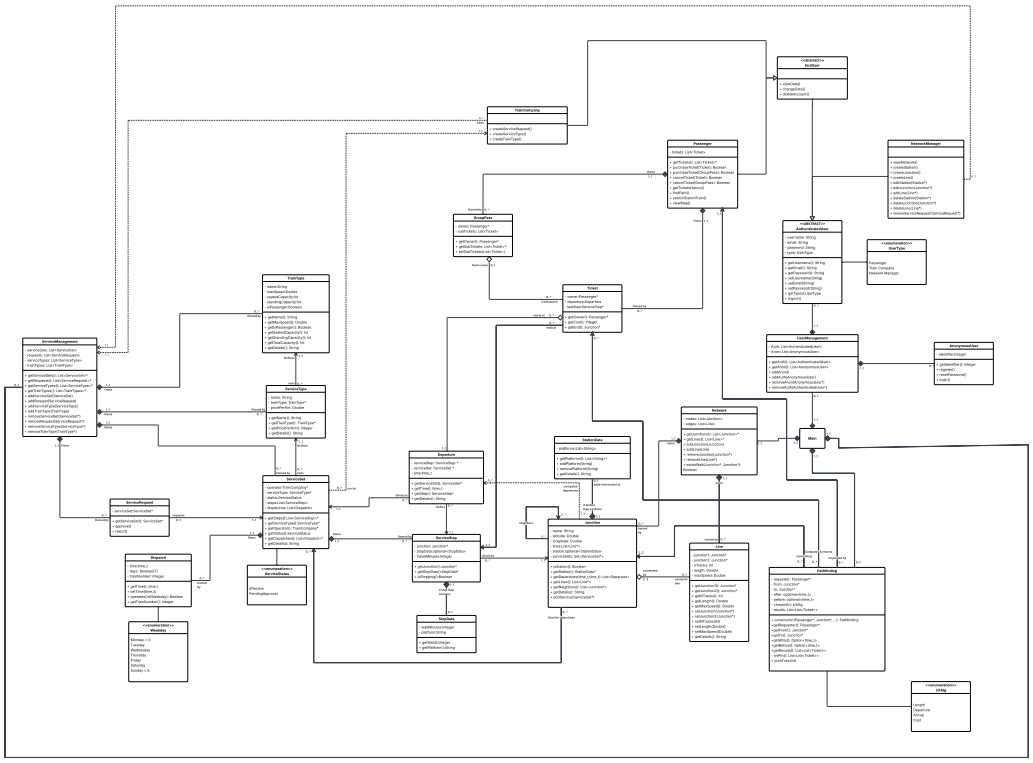
Use case	**Service Request Review**	
Release (id)	UC32, v0.1, 03/05/2026	
Context of Use	Service Management	
Scope	Let the Network Manager review a service request. FR27, FR28, FR29	
Level	Primary task	
Primary actor	Network Manager	
Stakeholder & interest	**Stakeholder**	**Interest**
	Train Company	Wants to review a service request
	System	Manages the user interface, the database and their interactions
Preconditions	The Network Manager is authenticated. There is a Service Request not yet reviewed.	
Minimal Guarantees	An invalid service request is denied.	
Success Guarantees	A service is approved, with possible amendment.	
Trigger	The Network Management initiates the review of a Service Request.	
Description	**Step**	**Action**
	1	The Train Company initiates the review of a Service Request.
	2	The system shows a list of registered Service Requests.
	3	The system asks the user to select a Service Request.
	4	The user selects a Service Request.
	5	The system performs automated service integrity checks.
	6	The system shows the results of the integrity checks.
	7	The system asks the user to select an operation between deny, amend and approve.
	8	If the user selects “deny”
	8.1	The system deletes the service request.
	9.1	If the user selects “approve”
	9.2	The system saves the service as effective.
	10	The system notifies the Train Company that requested the service.


UC32: Service Request Review
Network Manager
System
Initiate Service Request review
Show Service Requests list
Ask Service Request
Selects Service Request
Automated service integrity checks
Show check results
Ask operation
Insert operation
Deny
Approve
Delete Service Request
Save Service Request
Send Email
Send Email



UC32: Service Request Review

Class Diagram



Original image avaiable at our Github here

Description

Main: main class where all the management classes are instantiated

UserManagement: used to manage the user module

addAnon: Create an anonymous user with unique ID and add it to the list. Necessary for UC2, 7 addAuth: Upgrade an anonymous user into an authenticated one, removing the former and adding the latter to respective lists. Necessary for UC1, 6 removeAnon: Delete an anonymous user from list removeAutho: Remove an authenticated user from list. Necessary for UC2, 7

AnonymousUser: unregistered user who can access the system without authentication

register(): create a new account for UC1 resetPassword(): initiate the password reset process UC5 login(): become an authenticated user for UC6

AuthenticatedUser: authenticated user who uses the system

logout(): become an anonymous user for UC7

UserType: classification of the user, implements FR1

NetworkManager: authenticated user who manages the railway network, not an end user (in the following ¡Template¿ is a shortcut for Station, Junction or Line, used just for a short description)

viewNetwork(): view the network graph UC16 create¡Template¿(): create a new ¡Template¿ UC20, 21, 22 edit¡Template¿(¡Template¿): modify ¡Template¿ data UC23, 24, 25 delete¡Template¿(¡Template¿): remove a ¡Template¿ from the network UC26, 27 28 reviewServiceRequest(ServiceRequest): review a service request submitted by a train company UC32

EndUser: authenticated user who uses the end functionalities of the application

deleteAccount(): for UC2 viewData(): view personal account data UC3 changeData(): modify account data for UC4

Passenger: authenticated user who directly uses the train services

tickets: list of tickets owned by the passenger findPath(): initiate a path search between stations for UC12 searchStationTrain(): search for a train or station by name for UC14 viewMap(): view the network map for UC15 purchaseTicket(Ticket): buy an individual ticket for UC9, UC12.1 purchaseTicket(GroupPass): buy a group pass for UC9 cancelTicket(Ticket): cancel an individual ticket for UC10 cancelTicket(GroupPass): cancel a group pass for UC10 getTicketsHistory(): view purchased and cancelled tickets for UC11

TrainCompany: authenticated user who represents a train company createServiceRequest(): submit a new request for a service For UC31 createServiceType(): create a Service type For UC30 createTrainType(): create a Train type For UC29

Ticket: A representation of a passenger trip between two stations, it is used to represent possible trips as well as representing a booking.

owner: passenger who owns the ticket departure: The Departure this ticket starts from lastStep: the ServiceStep this ticket's validity ends getCost(): Compute the cost associated with this ticket (in cents) For UC8,UC9,UC10,UC12 getEnd(): Get the end junction for this ticket For UC8

GroupPass: group ticket

owner: passenger who bought the pass subTickets: list of tickets included in the pass

Pathfinding: A class of objects representing an in-use pathfinding search. The results of the search are stored each as a list of stations and the tickets

that would connect them

results: Storage of all results computed at construction, only accessed by srtPth() constructor(): Explicitly mentioned since it populates the internal fields and computes the possible paths, while other methods modify the results field as requested by the User. Sets the factory-default in chosenSrt. For UC12 srtPth(): Takes the paths and ranks them based on the enum parameter, filtering the top results nondestructively. Uses the srtAlg enum to change behaviour based on a pre-set list of supported sorting decisions. For UC12 getResults(): Unlike other getters, calls srtPth to only provide the processed results, without permanently altering the variable (so that the sorting can be repeated without loss of data). For UC12 and UC12.2 prchTcks(): Uses getTcks on the same selected path (by index int) to perform UC 12.1

srtAlg: The sorting algorithm to use

Length: Specified in FR14.1 Departure: Specified in FR14.2 Arrival: Specified in FR14.3 Cost: Specified in FR14.4

Network: used to represent the network module; graph representation of the railway infrastructure infrastructure

getJunctions(): get all junctions For UC13 ,UC15 getLines(): get all lines For UC13 ,UC15 addJunction(Junction): For UC21,UC22 addLine(Line): For UC22 removeJunction(Junction*): For UC26,27 removeLine(Line*): For UC 28 existsWalk: verify whether a path exists between two Junctions used in UC12

Line: undirected edge between two Junctions of the network, it's constructor implements UC19, 22

nTracks: number of tracks on the line maxSpeed: maximum speed trains can achieve getDetails(): provide a textual description of the object UC16

Junction: Junction of the railway network. If station data exists it is not just a simple junction but additionally a station. It's constructor implements UC18, 21

lines: The lines connected to this junction station: The optional station data associated to this junction serviceSets: A set of unique services that pass through or stop at this junction. This is used to compute departures. isStation(): True if this Junction is a station (I.E. It has station information) getStation(): Get the station data getDepartures(Time,Time): Compute and return a list of departures between given time criterion For UC13,UC12 getLines(): return a list of pointers to the connected lines getNeighbors(): return the neighboring Junctions via the connected lines getDetails(): provide a textual description of the object For UC16 addService(): Add a service that is

known to stop at this junction. For UC32

fStationData: Used to represent a station's data that may exist at a Junction (a Station is a Junction), It's constructor implements UC17, 20

platforms: A list of platforms addPlatform(String): Add a platform to the station if it doesn't already exist For UC24 removePlatform(String): Remove an existing platform For UC24 getDetails(): Provide a textual description for this element. For UC13,UC16

ServiceStep: one step of a service route. If stopData exists the service stops at this step

Junction: The Junction this step refers to stopData: The stopping information if this service does stop at this step travelMinutes: The time travelled isStopping(): If the service is stopping, true, otherwise false

Departure: A structure that holds a single departure's information

serviceSet: The ServiceSet that this departure references serviceStep: The step of the ServiceSet this departure refers to time: The time of the departure getDetails(): Provide a textual description for this element For UC13

StopData: stopping details for a service step

waitMinutes: minutes the service waits platform: from which platform the passengers access the train

ServiceManagement: used to manage the service module

ServiceRequest: request to introduce a service, its constructor implements UC31

serviceSet: serviceSet to be requested approve(): approve the requested service For UC32 reject(): reject the requested service For UC32

ServiceSet: a set of train services introduced by a train company along a common route

operator: train company who introduced the service steps: list of Junctions the service passes through serviceType: ServiceType used by the ServiceSet dispatches: A list of dispatches that define when services originate from the first serviceStep.

ServiceStatus: the status of a service

Effective: The service is operating PendingApproval: The service is awaiting approval

Dispatch: The dispatch information for an individual service

time: the time of departure from the first station in the ServiceSet days: The days of the week this service operates trainNumber: This service's globally unique identifier operatesOn(Weekday): Returns true if the service is operating on the given day of the week, otherwise false

ServiceType: valid type of train service, its constructor implements UC30

pricePerKm: price per km (in cents) getDetails(): returns a textual description of the object

TrainType: specification of a train model, its constructor implements UC29

maxSpeed: maximum speed the train can achieve isPassenger: true=passenger, false=cargo seatedCapacity: number of seats standingCapacity: number standing spots getDetails(): provide a textual description of the object